

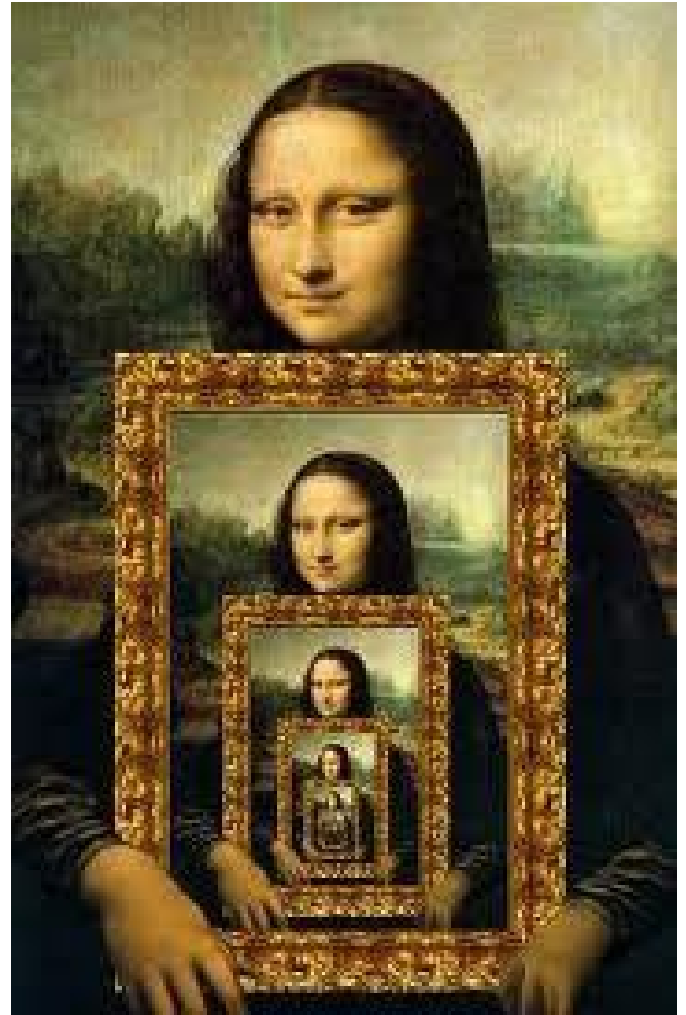
第2章 分治策略

(Divide and Conquer)

- 2.1 分治策略的基本思想
- 2.2 分治算法的分析技术
- 2.3 改进分治算法的途径
- 2.4 典型实例



递归



递归三要素

1. 递归终止条件 递归需要一个明确的临界点，程序一旦到达了
这个临界点，就不用继续往下递归，防止无限递归

2. 终止处理办法 在递归的临界点对应一种简单的情景，这种情
景下，应当直接给出问题的解决方案

3. 递归处理方法 递归问题必须可以分解为若干个规模较小、与
原问题 形式相同 的子问题，这些子问题可以用 相同的思路 来解决，并且合并得到原问题答案。

递归 – 阶乘

阶乘计算：从键盘输入正整数N（ $0 \leq N \leq 20$ ），输出N!。

$$N! = \begin{cases} 1 & N = 0 \\ N(N-1)! & N > 0 \end{cases}$$

边界条件

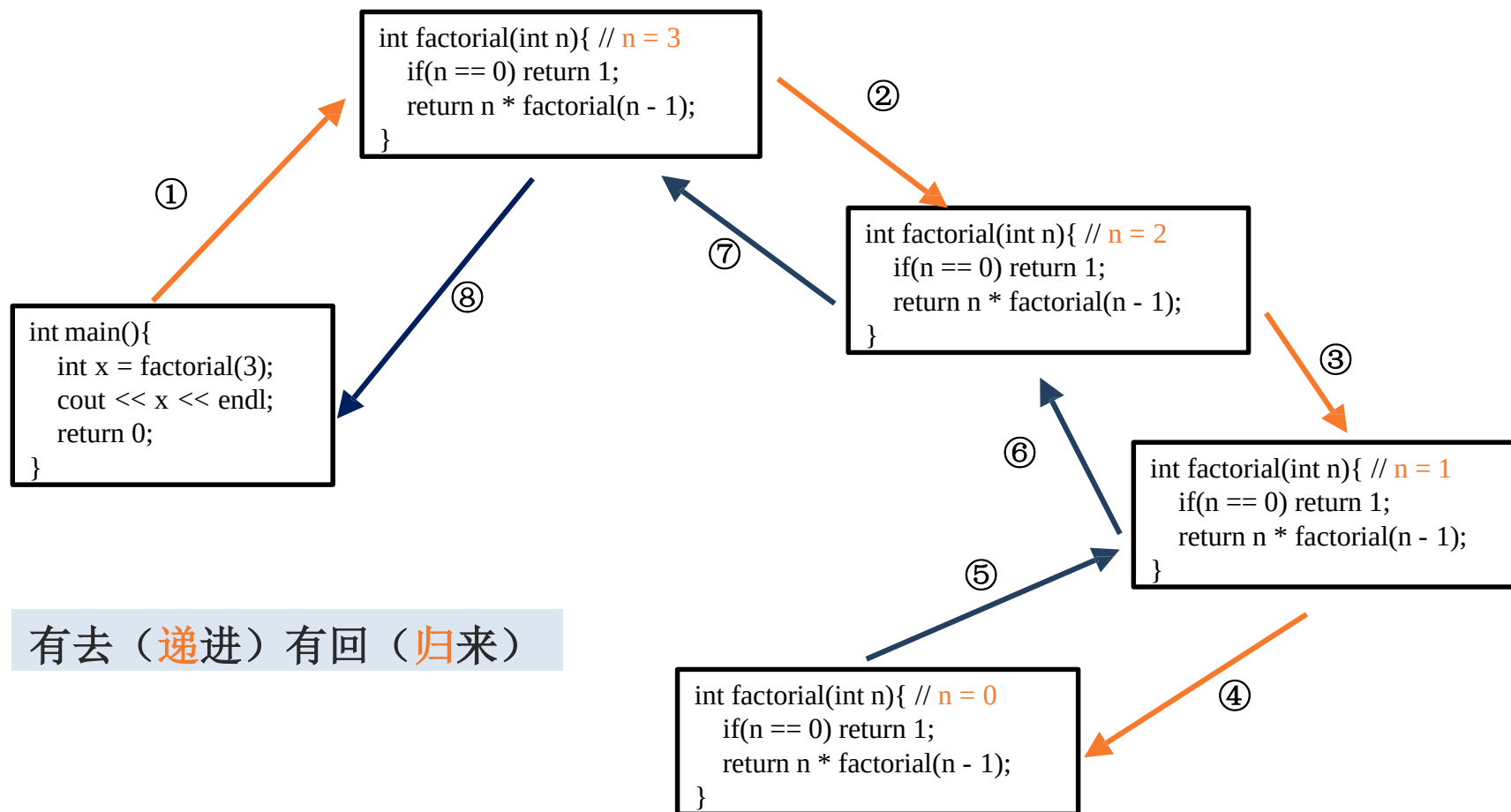
递归方程

```
int factorial(int n) {  
    if(n == 0) return 1; //递归边界  
    return n*factorial(n-1);  
}
```

递归出口

递归调用

阶乘 - 递归调用过程



有去（递进）有回（归来）

二分检索

算法 Binary Search (T, l, r, x)

输入：数组 T ，下标从 l 到 r ；数 x

输出： j // 若 x 在 T 中, j 为下标；否则为 0

1. $l \leftarrow 1; r \leftarrow n$
2. while $l \leq r$ do
3. $m \leftarrow \lfloor (l+r)/2 \rfloor$ // m 为中间位置
4. if $T[m] = x$ then return m // x 是中位数
5. else if $T[m] > x$ then $r \leftarrow m-1$
6. else $l \leftarrow m+1$
7. return 0

二分检索算法设计思想

- 通过 x 与中位数的比较，将原问题归结为规模减半的子问题，如果 x 小于中位数，则子问题由小于 x 的数构成，否则子问题由大于 x 的数构成。
- 对子问题进行二分检索。
- 当子问题规模为 1 时，直接比较 x 与 $T[m]$ ，若相等则返回 m ，否则返回 0。



是否能够递归实现？

二分检索时间复杂度分析

二分检索问题最坏情况下时间复杂度

$$W(n) = W(\lfloor n/2 \rfloor) + 1$$

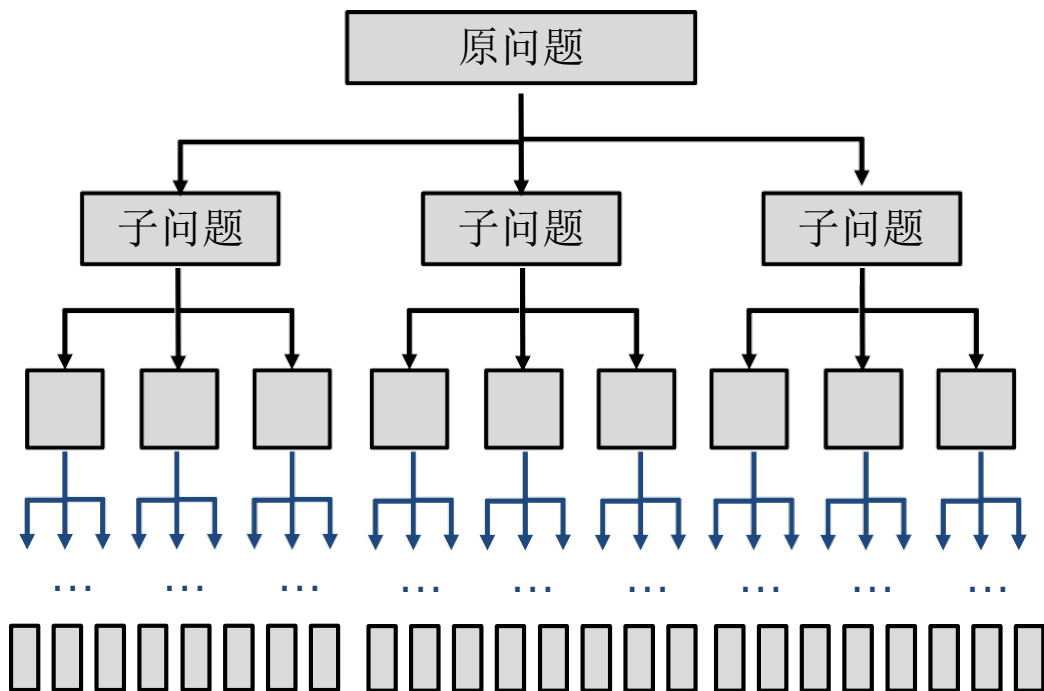
$$W(1) = 1$$

可以解出

$$W(n) = \lfloor \log n \rfloor + 1$$

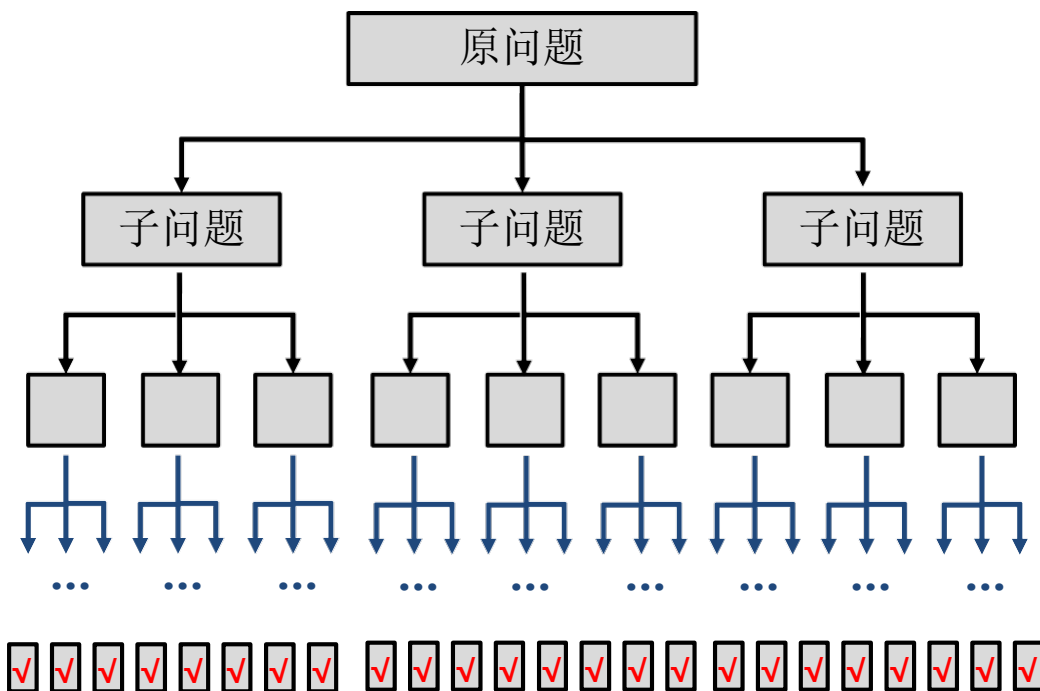
分治算法基本思想

分 — 将大规模的原问题分割成 k 个更小规模的 **子问题**，如果子问题的规模仍然不够小，则再划分为 k 个“**子 子**”问题，如此**递归地**进行下去，直到子问题规模足够小（**基础问题**），很容易求出其解为止。



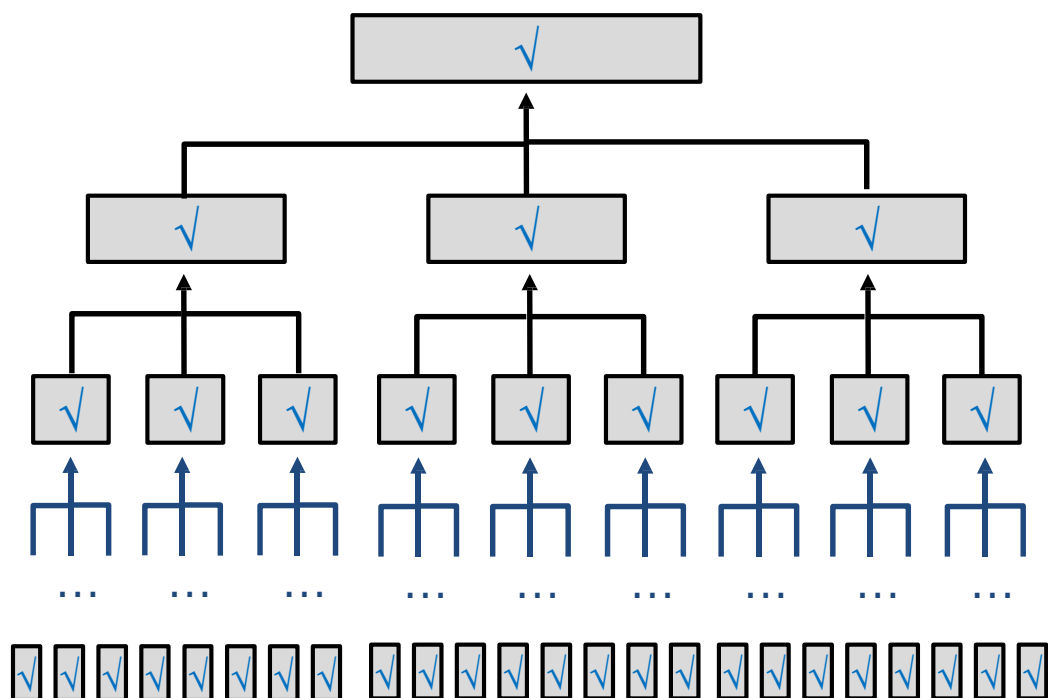
分治算法基本思想

治 — 求解规模足够小的基础问题。



分治算法基本思想

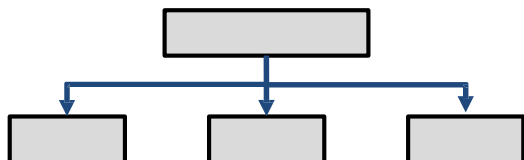
合 — 将求出的小规模的问题的解合并为一个更大规模的问题的解，**自底向上**逐步求出原来问题的解。



分治算法基本思想

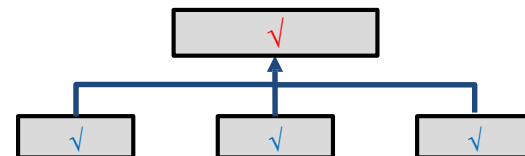
分治策略算法细分为三个阶段： Divide 、 Conquer 、 Combine 。 Divide 阶段是把原问题分割成小问题， Conquer 阶段是递归处理流程， Combine 阶段是运用小问题的答案合成出原问题的解答。

Divide

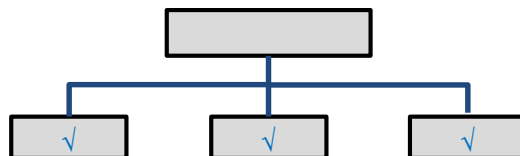


“分治合” 策略

Combine



Conquer



分治策略框架程序

```
divide-and-conquer(P){
```

```
(1)  if ( | P | <= n0) adhoc(P); //递归出口，用特定程序解决基础问题
```

```
(2)  divide P into smaller subinstances P1,P2,...,Pk; //分解出子问题
```

```
(3)  for (i = 1, i <= k, i++)
```

```
        yi = divide-and-conquer(Pi); //递归求解各子问题
```

```
(4)  return merge(y1,...,yk); //将各子问题的解合并为原问题的解 }
```

□ $|P|$ 表示问题P的规模， n_0 是一个预先定义的阈值，表示当问题P的规模不超过 n_0 时，**问题已容易求解**，不必要继续分解和递归调用。

□ **adhoc(P)**是分治算法中的子程序，对应**治过程**（或者说**conquer**），一般是常数时间复杂度的子函数或者子过程。

分治策略框架程序

```
divide-and-conquer(P){
```

```
(1)  if ( | P | <= n0) adhoc(P); //递归出口，用特定程序解决基础问题
```

```
(2)  divide P into smaller subinstances P1,P2,...,Pk; //分解出子问题
```

```
(3)  for (i = 1, i <= k, i++)
```

```
        yi = divide-and-conquer(Pi); //递归求解各子问题
```

```
(4)  return merge(y1,...,yk); //将各子问题的解合并为原问题的解 }
```

设计划分策略，把原问题**P**分解成**k**个规模较小的子问题，这个步骤是分治算法的基础和关键，人们往往遵循两个原则：

- **平衡子问题原则**，分割出的**k**个子问题其规模最好大致相当；
- **独立子问题原则**，分割出的**k**个子问题之间重叠越少越好，最好**k**个子问题是相互独立，不存在重叠子问题。

分治策略基本原理

divide-and-conquer(P){

(1) if ($|P| \leq n_0$) adhoc(P); //递归出口，用特定程序解决基础问题

(2) divide P into smaller subinstances $P_1, P_2, \dots, P_k$; //分解出子问题

(3) for ($i = 1, i \leq k, i++$)

$y_i = \text{divide-and-conquer}(P_i)$; //递归求解各子问题

(4) return **merge**($y_1, \dots, y_k$); //将各子问题的解合并为原问题的解 }

merge合并子程序，把**k**个子问题的解合并得到原问题的解。合并子程序因求解问题而异，即使是同一个问题，如果第二步的划分策略不同，其合并子程序也往往不一样。

设计要点

- 原问题可以划分或者归约为规模较小的子问题
 - 子问题与原问题具有相同的性质
 - 子问题的求解彼此独立
 - 划分时子问题的规模尽可能均衡
- 子问题规模足够小时可直接求解
- 子问题的解综合得到原问题的解
- 算法实现：递归或迭代

分治算法时间分析

时间复杂度函数的递推方程

$$W(n) = W(|P_1|) + W(|P_2|) + \dots + W(|P_k|) + f(n)$$

$$W(c) = C$$

- $P_1, P_2, \dots, P_k$ 为 划分后产生的 子问题
- $f(n)$ 为划分子问题以及将子问题的解综合得到原问题解的总工作量
- 规模为 c 的最小子问题的的工作量为 C

二分归并排序

算法 Merge Sort (A, p, r)

输入：数组 $A[p..r]$

输出：元素按从小到大排序的数组 A

1. if $p < r$
2. then $q \leftarrow \lfloor (p + r)/2 \rfloor$ 对半划分
3. Merge Sort (A, p, q) 子问题1
4. Merge Sort ($A, q+1, r$) 子问题 2
5. Merge (A, p, q, r) 综合解

二分归并排序设计思想

- 划分将原问题归结为规模为 $n/2$ 的 2 个子问题
- 继续划分，将原问题归结为规模为 $n/4$ 的 4 个子问题. 继续...，当子问题规模为1 时，划分结束.
- 从规模 1 到 $n/2$ ，陆续归并被排好序的两个子数组. 每归并一次，数组规模扩大一倍，直到原始数组.

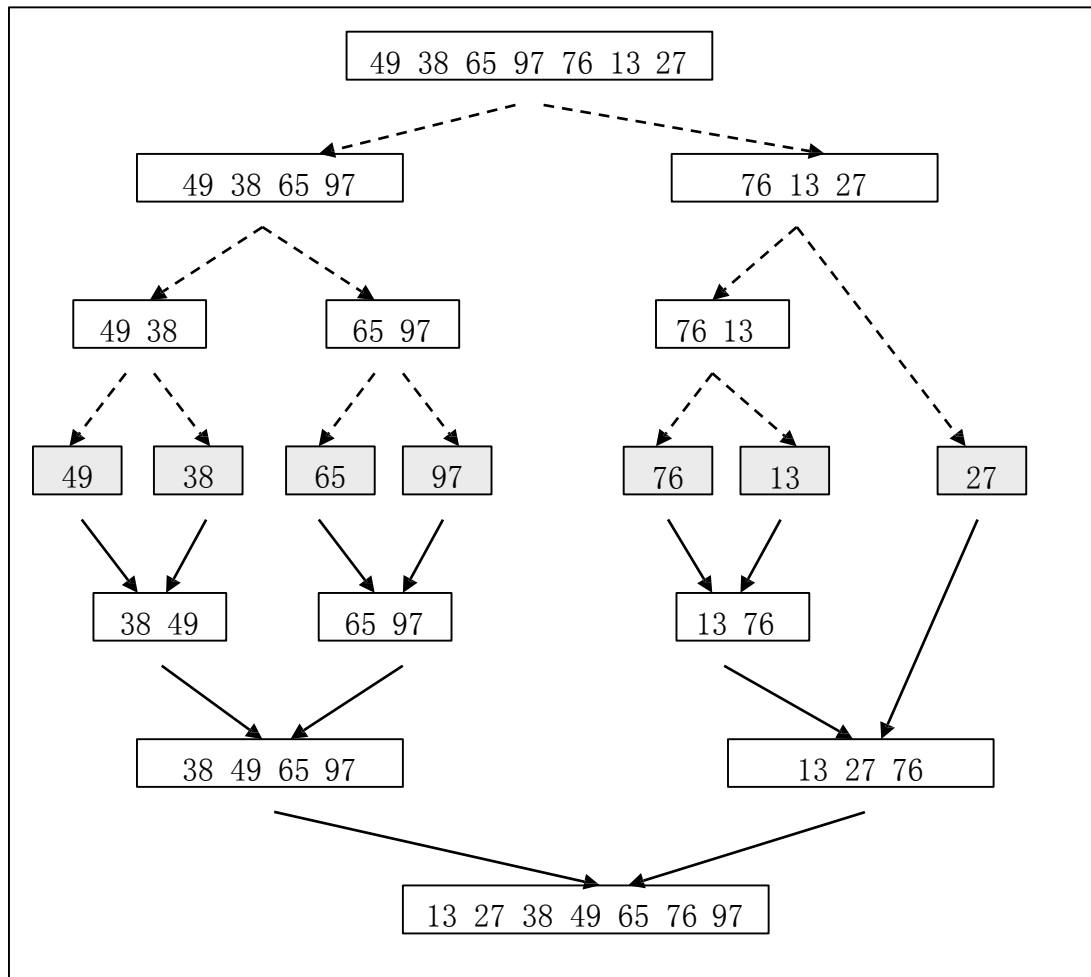
合并排序基本思想

分：根据整数集合的规模把原始的整数集合（记为 $A = \{a[l], \dots, a[r]\}$ ）平均分成两部分： $A_1 = \{a[l], \dots, a[(l+r)/2]\}$ 元素为第一部分， $A_2 = \{a[(l+r)/2 + 1], \dots, a[r]\}$

治：如果划分后的子集 A_1 和 A_2 只包含一个整数，则不需任何操作，把这单个整数当成已排好序的集合；否则，把该子集继续分割，然后递归调用。

合：设计子程序merge把两个已经升序排列的子集 B_1 ， B_2 合并成一个整体升序排列的集合 B 。

合并排序总体过程描述



二分归并排序时间复杂度分析

假设 n 为2的幂，二分归并排序最坏情况下时间复杂度

$$W(n) = 2W(n/2) + n - 1$$

$$W(1) = 0$$

可以解出

$$W(n) = n \log n - n + 1$$

两类常见的递推方程

$$f(n) = \sum_{i=1}^k a_i f(n-i) + g(n) \quad (1)$$

$$f(n) = af\left(\frac{n}{b}\right) + d(n) \quad (2)$$

例子:

Hanoi塔, $W(n) = 2W(n-1) + 1$

二分检索, $W(n) = W(n/2) + 1$

归并排序, $W(n) = 2W(n/2) + n - 1$

递推方程的求解

方程1
$$f(n) = \sum_{i=1}^k a_i f(n-i) + g(n)$$

方程2
$$f(n) = af\left(\frac{n}{b}\right) + d(n)$$

求解方程：

第一类方程：迭代法、递归树、尝试法

第二类方程：迭代法、换元法、递归树、主定理

方程2的解

方程 $T(n) = aT(n/b) + d(n)$

$d(n)$ 为常数

$$T(n) = \begin{cases} O(n^{\log_b a}) & a \neq 1 \\ O(\log n) & a = 1 \end{cases}$$

$d(n) = cn$

$$T(n) = \begin{cases} O(n) & a < b \\ O(n \log n) & a = b \\ O(n^{\log_b a}) & a > b \end{cases}$$

逆序对问题

问题描述： 设 $A[1, \dots, n]$ 是一个包含 n 个不同非负整数的数组。如果在 $i < j$ 的情况下，有 $A[i] > A[j]$ ，则 $(A[i], A[j])$ 就称为 A 中的一个逆序对。例如，数组 $(3, 1, 4, 5, 2)$ 的“逆序对”有 $\langle 3,1 \rangle$ 、 $\langle 3,2 \rangle$ 、 $\langle 4,2 \rangle$ 、 $\langle 5,2 \rangle$ 共4个。

输入： 每个测试用例包括两行，第一行输入整数的个数 n ， $n \leq 10000$ ，第二行输入 n 个整数，数与数之间用空格隔开。最后一行输入-1，表示输入结束。

输出： 每组测试数据的结果输出占一行，输出该数组中逆序对的个数。

样例输入：

```
5
3 1 4 5 2
-1
```

样例输出：

```
4
```

问题分析

■ 枚举算法

时间复杂度是 $O(n^2)$

■ 小规模问题

当数组中元素的个数 n 比较少时，容易统计逆序对的数目

- 数组中只有1个元素的话，逆序对的数目为0；
- 数组中有2个元素，如果第一个元素大于第二个元素，则有1个逆序对，否则0个逆序对。
- 数组中元素个数超过2，怎么办？

算法思路

分：根据整数集合的规模把原始的整数集合 A （记为 $A = \{a[l], \dots, a[r]\}$ ）平均分成两部分： $A_1 = \{a[l], \dots, a[(l+r)/2]\}$ 和 $A_2 = \{a[(l+r)/2 + 1], \dots, a[r]\}$ 。

治：如果分割后的子数组只包含1个整数，显然，该子数组中没有逆序对，返回0；否则，把该子集继续分解，并递归调用。

合：原数组 A 中的逆序对可以分为三部分：第一部分为子数组 A_1 中的逆序对，其数目记为**C1**；第二部分为子数组 A_2 中的逆序对，其数目记为**C2**；第三部分是一个元素在 A_1 中，而另外一个元素在 A_2 中的逆序对 $(A[i], A[j])$ ，其中 $A[i] \in A_1, A[j] \in A_2$ ，其数目记为**C3**。

算法思路

$$C = C1 + C2 + C3$$

怎么计算**C3**?

1) 枚举所有的元素对 $(A[i], A[j])$ ，其中 $A[i] \in A_1, A[j] \in A_2$ ，判断 $(A[i], A[j])$ 是否是逆序对。总共需要判断 $(n/2)^2 = n^2/4$ 个元素对，其时间复杂度为 $O(n^2)$ 。

2) 如果在求解数组 A 的逆序对之前，把子数组 A_1 和 A_2 都按照升序排列，则求 $C3$ 时只需要顺序扫描子数组 A_1 和 A_2 一次即可计算，其时间复杂度为 $O(n)$ 。

算法思路

给定升序排列的两个子数组，怎样计算 C_3 ？



$38 > 13 \implies \{49, 65, 97\} > 13 \implies$ 增加4个逆序对

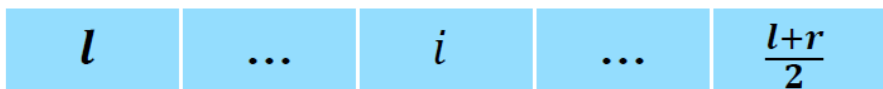
$38 < 40 \implies 38 < \{76\} \implies$ 38不会再与A2中元素产生逆序对

$49 > 40 \implies \{65, 97\} > 40 \implies$ 增加3个逆序对

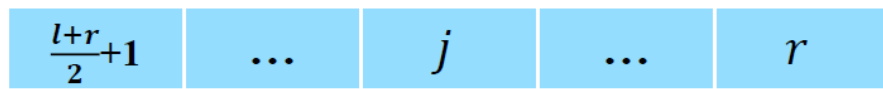
$49 < 76 \implies$ 49不会再与A2中元素产生逆序对

$65 < 76 \implies$ 65不会再与A2中元素产生逆序对

$97 > 76 \implies$ 增加1个逆序对



A1



A2

1) 如果 $A_1[i] > A_2[j]$, 则有:

$$A_1[i+1] > A_2[j];$$

.....

$$A_1[(l+r)/2] > A_2[j];$$

一次比较可以推导出多个逆序对

$$C3 += (l + r)/2 - i + 1$$

$$j++$$

2) 如果 $A_1[i] \leq A_2[j]$, 则有:

$$A_1[i] \leq A_2[j+1];$$

.....

$$A_1[i] \leq A_2[r];$$

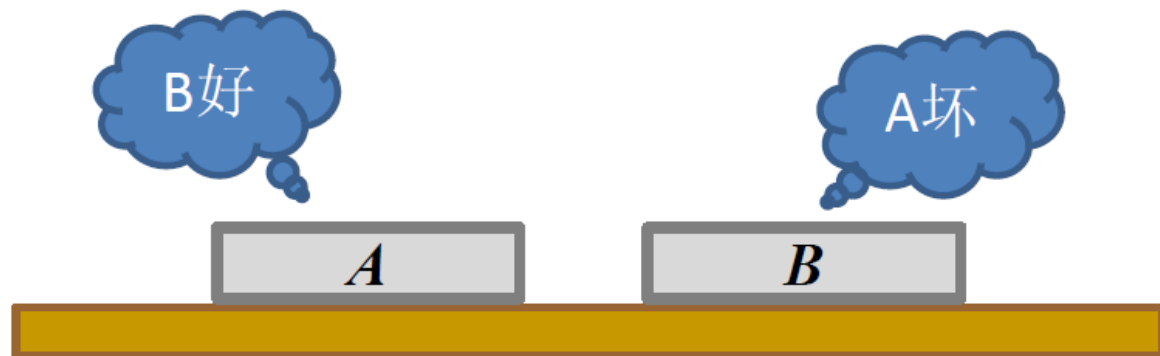
一次比较可以省略多次比较

C3 保持不变

$$i++$$

一次测试过程

测试方法：将2片芯片（ A 和 B ）置于测试台上，互相进行测试，测试报告是“好”或“坏”，只取其一。



假设：好芯片的报告一定是正确的，坏芯片的报告是不确定的（可能会出错）

问题

输入：

n 片芯片，其中好芯片至少比坏芯片多 1 片。

问题：

设计一种测试方法，通过测试从 n 片芯片中挑出 1 片好芯片。

要求：使用最少的测试次数。

测试结果分析

<i>A</i> 报告	<i>B</i> 报告	结论
<i>B</i> 是好的	<i>A</i> 是好的	<i>A, B</i> 都好或 <i>A, B</i> 都坏
<i>B</i> 是好的	<i>A</i> 是坏的	至少一片是坏的
<i>B</i> 是坏的	<i>A</i> 是好的	至少一片是坏的
<i>B</i> 是坏的	<i>A</i> 是坏的	至少一片是坏的

判定芯片 A 的好坏

问题：给定芯片 A ，判定 A 的好坏

方法：用其他 $n-1$ 片芯片对 A 测试.

$n=7$ ：好芯片数 ≥ 4 .

A 好，6个报告中至少 3 个报“好”

A 坏，6个报告中至少 4 个报“坏”

n 是奇数：好芯片数 $\geq (n+1)/2$.

A 好，至少有 $(n-1)/2$ 个报“好”

A 坏，至少有 $(n+1)/2$ 个报告“坏”

结论：至少一半报“好”， A 是好芯片，
超过一半报“坏”， A 是坏芯片.

判定芯片A的好坏

$n=8$: 好芯片数 ≥ 5 .

A 好, 7个报告中至少 4 个报“好”

A 坏, 7个报告中至少 5 个报“坏”

n 是偶数: 好芯片数 $\geq n/2+1$.

A 好, 至少有 $n/2$ 个报告“好”

A 坏, 至少有 $n/2+1$ 个报告“坏”

结论: $n-1$ 份报告中,
至少一半报“好”, 则 A 为好芯片
超过一半报“坏”, 则 A 为坏芯片

蛮力算法

测试方法：任取 1 片测试，如果是好芯片，测试结束；如果是坏芯片，抛弃，再从剩下芯片中任取 1 片测试，直到得到 1 片好芯片。

时间估计： ?

$$\Theta(n^2)$$

分治算法设计思想

假设 n 为偶数，将 n 片芯片两两一组做测试淘汰，剩下芯片构成子问题，进入下一轮分组淘汰。

淘汰规则：

"好, 好" $\Rightarrow$ 任留 1 片，进入下轮
其他情况 $\Rightarrow$ 全部抛弃

递归截止条件： $n \leq 3$

3 片芯片，1 次测试可得到好芯片。

1 或 2 片芯片，不再需要测试。

分治算法的正确性

命题1 当 n 是偶数时，在上述淘汰规则下，经过一轮淘汰，剩下的好芯片比坏芯片至少多1片.

证 设 A, B 都好的芯片 i 组, A 与 B 一好一坏 j 组, A 与 B 都坏的 k 组. 淘汰后好芯片至少 i 片, 坏芯片至多 k 片.

$$2i + 2j + 2k = n \quad \text{初始芯片总数}$$

$$2i + j > 2k + j \quad \text{初始 好芯片多于坏芯片}$$

 $i > k$

n 为奇数时的特殊处理

当 n 是奇数时，可能出问题

输入：好 好 好 好 坏 坏 坏

分组：好 | 好 好 | 好 坏 | 坏 坏

淘汰后：好 好 坏 坏

处理办法：当 n 为奇数时，增加一轮对轮空芯片的单独测试。

如果该芯片为好芯片，算法结束；
如果是坏芯片，则淘汰该芯片。

伪码描述

算法 Test(n)

1. $k \leftarrow n$

分组
淘汰

2. while $k > 3$ do

3. 将芯片分成 $\lfloor k/2 \rfloor$ 组 // 轮空处理

4. for $i = 1$ to $\lfloor k/2 \rfloor$ do

5. if 2片好 then 则任取1片留下

6. else 2片同时丢掉

7. $k \leftarrow$ 剩下的芯片数

8. if $k = 3$ then

递归
结束

9. 任取2片芯片测试

10. if 1好1坏 then 取没测的芯片

11. else 任取1片被测芯片

12. if $k = 2$ or 1 then 任取1片

时间复杂度分析

设输入规模为 n

每轮淘汰后，芯片数至少减半

测试次数(含轮空处理): $O(n)$

时间复杂度:

$$W(n) = W(n/2) + O(n)$$

$$W(3) = 1, W(2) = W(1) = 0$$

解得 $W(n) = O(n)$

快速排序基本思想

- 用首元素 x 作划分标准，将输入数组 A 划分成不超过 x 的元素构成的数组 A_L ，大于 x 的元素构成的数组 A_R 。其中 A_L, A_R 从左到右存放在数组 A 的位置。
- 递归地对子问题 A_L 和 A_R 进行排序，直到子问题规模为 1 时停止。

算法思路

分：首先考虑怎样对原始的整数集合（记为 $A = \{a[l], \dots, a[r]\}$ ）进行划分。以元素 $a[l]$ 为基准元素，把原数组 A 分为三个部分： $A_1 = \{a'[l], \dots, a'[k-1]\}$ ， $A_2 = \{a'[k]\}$ ，和 $A_3 = \{a'[k+1], \dots, a'[r]\}$ ，其中 A_1 中所有元素的值都小于等于基准元素 $a[l]$ ，而 A_3 中所有元素的值都大于基准元素 $a[l]$ ， A_2 中元素 $a'[k]$ 就是调整位置后的基准元素 $a[l]$ 。

治：如果划分后的子集 A_1 和 A_3 只包含一个整数，则不需任何操作，把这单个整数当成已排好序的集合；否则，把该子集继续分割，然后递归调用。子集 A_2 中元素划分后其位置保持不变。

合：因为子集 A_1 和 A_3 的排序是递归调用的，所以，在 A_1 和 A_3 都已经排好序后，不需要执行任何操作， A_1 ， A_2 和 A_3 自然排列就是有序的。

伪码

算法 Quicksort (A, p, r)

输入：数组 $A[p..r]$

输出：排好序的数组 A

1. if $p < r$
2. then $q \leftarrow \text{Partition}(A, p, r)$
3. $A[p] \leftrightarrow A[q]$
4. Quicksort ($A, p, q-1$)
5. Quicksort ($A, q+1, r$)

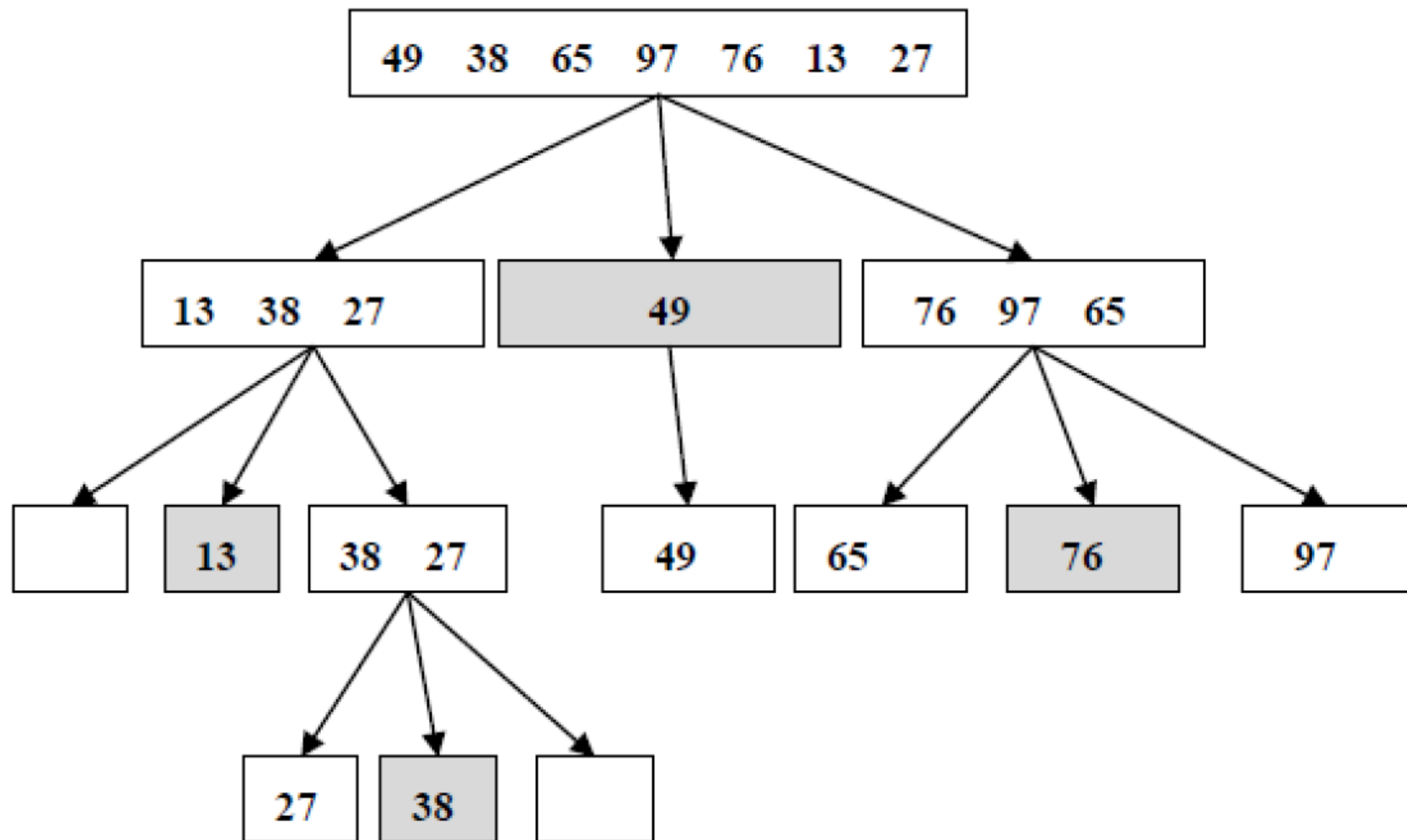
初始置 $p=1, r=n$ ，然后调用上述算法

划分过程

Partition (A, p, r)

1. $x \leftarrow A[p]$
2. $i \leftarrow p$
3. $j \leftarrow r + 1$
4. while true do
5. repeat $j \leftarrow j - 1$
6. until $A[j] \leq x$ // 不超过首元素的
7. repeat $i \leftarrow i + 1$
8. until $A[i] > x$ // 比首元素大的
9. if $i < j$
10. then $A[i] \leftrightarrow A[j]$
11. else return j

算法示例



划分实例

27 **99** 0 8 13 64 86 16 7 10 88 **25** 90
i *j*

27 25 0 8 13 **64** 86 16 7 **10** 88 99 90
i *j*

27 25 0 8 13 10 **86** 16 **7** 64 88 99 90
i *j*

27 25 0 8 13 10 7 **16** **86** 64 88 99 90
j *i*

16 25 0 8 13 10 7 **27** 86 64 88 99 90

时间复杂度

最坏情况: $W(n) = W(n-1) + n - 1$

$$W(1) = 0$$

$$W(n) = n(n-1)/2$$

最好划分: $T(n) = 2 T(n/2) + n - 1$

$$T(1) = 0$$

$$T(n) = \Theta(n \log n)$$

均衡划分的时间复杂度

均衡划分：子问题的规模比不变
例如为 1:9

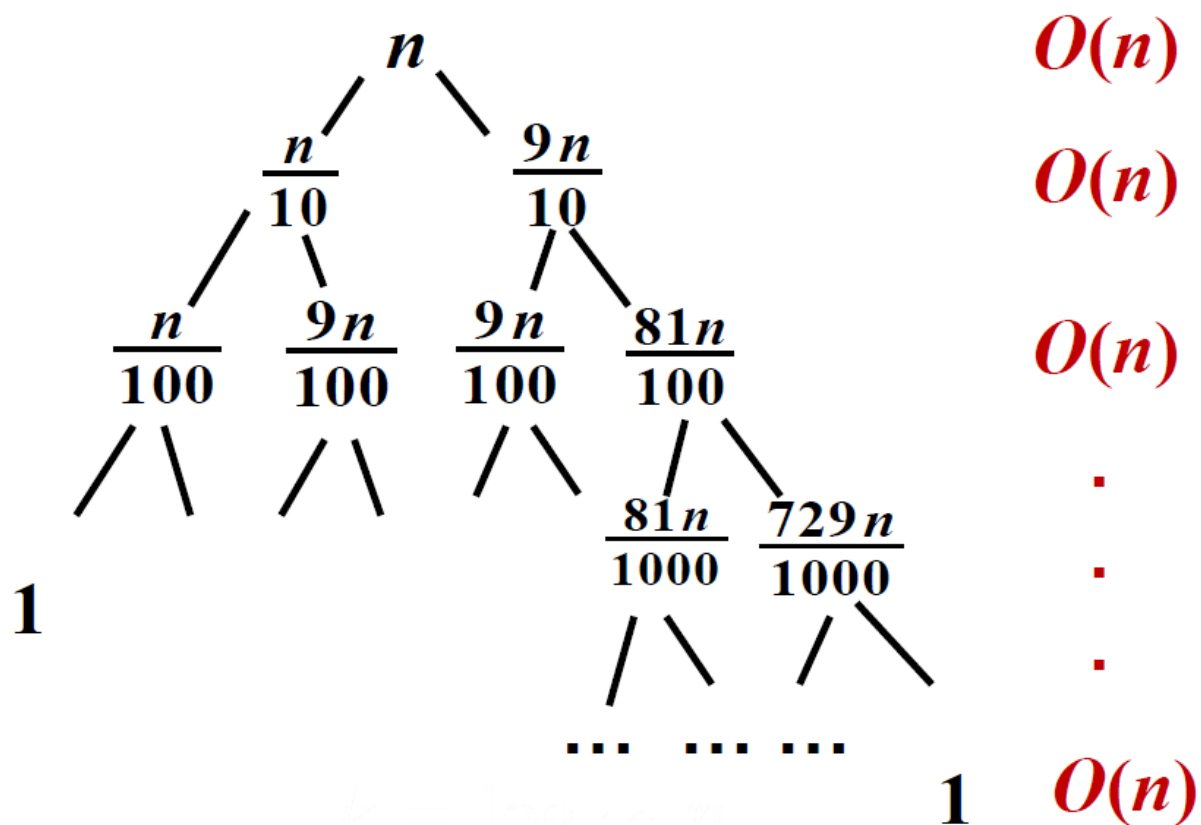
$$T(n) = T(n/10) + T(9n/10) + n$$

$$T(1) = 0$$

根据递归树，时间复杂度

$$T(n) = \Theta(n \log n)$$

递归树



$$T(n) = O(n \log n)$$

平均时间复杂度

首元素排好序后处在 $1, 2, \dots, n$

各种情况概率都为 $1/n$

首元素在位置 1: $T(0), T(n-1)$

首元素在位置 2: $T(1), T(n-2)$

....

首元素在位置 $n-1$: $T(n-2), T(1)$

首元素在位置 n : $T(n-1), T(0)$

子问题工作量 $2[T(1)+T(2)+\dots+T(n-1)]$

划分工作量 $n-1$

平均时间复杂度

$$T(n) = \frac{1}{n} \sum_{k=1}^{n-1} (T(k) + T(n-k)) + n - 1$$

$$T(n) = \frac{2}{n} \sum_{k=1}^{n-1} T(k) + n - 1$$

$$T(1) = 0$$

$$T(n) = \Theta(n \log n)$$

首元素划分后每个位置概率相等

幂乘问题

输入： a 为给定实数， n 为自然数

输出： a^n

传统算法： 顺序相乘

$$a^n = (\dots(((a \ a)a)a)\dots)a$$

乘法次数： $\Theta(n)$

分治算法——划分

n 为偶数 $\underbrace{a \dots a}_{n/2 \text{ 个}} \mid \underbrace{a \dots a}_{n/2 \text{ 个}}$

n 为奇数 $\underbrace{a \dots a}_{(n-1)/2 \text{ 个}} \mid \underbrace{a \dots a}_{(n-1)/2 \text{ 个}} \mid a$

$$a^n = \begin{cases} a^{n/2} \times a^{n/2} & n \text{ 为偶数} \\ a^{(n-1)/2} \times a^{(n-1)/2} \times a & n \text{ 为奇数} \end{cases}$$

分治算法分析

以乘法作为基本运算

- 子问题规模：不超过 $n/2$
- 两个规模近似 $n/2$ 的子问题完全一样，只要计算1次

$$W(n) = W(n/2) + \Theta(1)$$

$$W(n) = \Theta(\log n)$$

幂乘算法的应用

Fibonacci数列: 1, 1, 2, 3, 5, 8, 13, 21, ...

增加 $F_0=0$, 得到数列

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

问题: 已知 $F_0=0, F_1=1$, 给定 n , 计算 F_n .

通常算法: 从 $F_0, F_1, \dots$ 开始, 根据递推公式

$$F_n = F_{n-1} + F_{n-2}$$

陆续相加可得 F_n , 时间复杂度为 $\Theta(n)$

Fibonacci数的性质

定理1 设 $\{F_n\}$ 为 Fibonacci 数构成的数列，那么

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

归纳证明

$$n=1, \text{ 左边} = \begin{bmatrix} F_2 & F_1 \\ F_1 & F_0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \text{右边}$$

Fibonacci数的性质(续)

假设对任意正整数 n , 命题成立, 即

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

那么 $\begin{bmatrix} F_{n+2} & F_{n+1} \\ F_{n+1} & F_n \end{bmatrix} = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$

$$= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n+1}$$

归纳假
设代入

算法

令矩阵 $M = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$, 用乘幂算法计算 M^n

时间复杂度:

- 矩阵乘法次数 $T(n) = \Theta(\log n)$
- 每次矩阵乘法需要做 8 次元素相乘
- 总计元素相乘次数为 $\Theta(\log n)$

改进分治算法的途径1:减少子问题数

减少子问题个数的依据

分治算法的时间复杂度方程

$$W(n) = aW(n/b) + d(n)$$

a : 子问题数, n/b : 子问题规模,

$d(n)$: 划分与综合工作量.

当 a 较大, b 较小, $d(n)$ 不大时, 方程的解:

$$\underline{W(n) = \Theta(n^{\log_b a})}$$

减少 a 是降低函数 $W(n)$ 的阶的途径.

利用子问题的依赖关系, 使某些子问题的解通过组合其他子问题的解而得到.

例1：整数位乘问题

输入： X, Y 是 n 位二进制数， $n = 2^k$

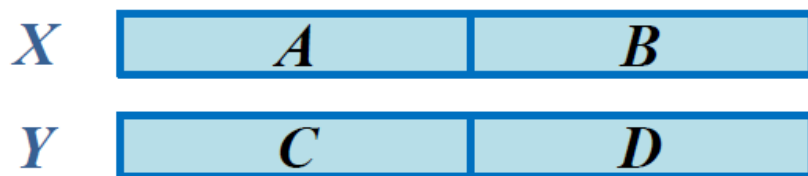
输出： XY

普通乘法： 需要 $O(n^2)$ 次位乘运算

简单划分： 令

$$X = A2^{n/2} + B, \quad Y = C2^{n/2} + D.$$

$$XY = \underline{AC} 2^n + (\underline{AD} + \underline{BC}) 2^{n/2} + \underline{BD}$$



$$W(n) = 4W(n/2) + O(n) \Rightarrow W(n) = O(n^2)$$

减少子问题个数

子问题间的依赖关系：代数变换

$$AD+BC = (\underline{A-B})(\underline{D-C}) + \underline{AC} + \underline{BD}$$

算法复杂度

$$W(n) = 3 W(n/2) + cn$$

$$W(1) = 1$$

方程的解

$$W(n) = O(n^{\log 3}) = O(n^{1.59})$$

例2：矩阵相乘问题

输入： A, B 为 n 阶矩阵， $n = 2^k$

输出： $C = AB$

通常矩阵乘法：

C 中有 n^2 个元素

每个元素需要做 n 次乘法

以元素相乘为基本运算

$$W(n) = O(n^3)$$

简单分治算法

分治法 将矩阵分块，得

$$\begin{pmatrix} \boxed{A_{11}} & \boxed{A_{12}} \\ \boxed{A_{21}} & \boxed{A_{22}} \end{pmatrix} \begin{pmatrix} \boxed{B_{11}} & B_{12} \\ \boxed{B_{21}} & B_{22} \end{pmatrix} = \begin{pmatrix} \boxed{C_{11}} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

其中

$$\begin{aligned} C_{11} &= \underline{A_{11}B_{11}} + \underline{A_{12}B_{21}} & C_{12} &= \underline{A_{11}B_{12}} + \underline{A_{12}B_{22}} \\ C_{21} &= \underline{A_{21}B_{11}} + \underline{A_{22}B_{21}} & C_{22} &= \underline{A_{21}B_{12}} + \underline{A_{22}B_{22}} \end{aligned}$$

递推方程 $W(n) = 8 W(n/2) + cn^2$

$$W(1) = 1$$

解

$$W(n) = O(n^3).$$

Strassen 矩阵乘法

变换方法:

设计 $M_1, M_2, \dots, M_7$, 对应7个子问题

$$M_1 = A_{11} (B_{12} - B_{22})$$

$$M_2 = (A_{11} + A_{12}) B_{22}$$

$$M_3 = (A_{21} + A_{22}) B_{11}$$

$$M_4 = A_{22} (B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{22}) (B_{11} + B_{22})$$

$$M_6 = (A_{12} - A_{22}) (B_{21} + B_{22})$$

$$M_7 = (A_{11} - A_{21}) (B_{11} + B_{12})$$

Strassen 矩阵乘法(续)

利用中间矩阵，得到结果矩阵

$$C_{11} = M_5 + M_4 - M_2 + M_6$$

$$C_{12} = M_1 + M_2$$

$$C_{21} = M_3 + M_4$$

$$C_{22} = M_5 + M_1 - M_3 - M_7$$

时间复杂度函数：

$$W(n) = 7 W(n/2) + 18(n/2)^2$$

$$W(1) = 1$$

解 $W(n) = O(n^{\log 7}) = O(n^{2.8075})$

矩阵乘法的研究及应用

矩阵乘法问题的难度：

- Coppersmith–Winograd算法： $O(n^{2.376})$
目前为止最好的上界
- 目前为止最好的下界是： $\Omega(n^2)$

应用：

- 科学计算、图像处理、数据挖掘等
- 回归、聚类、主成分分析、决策树等挖掘算法常涉及大规模矩阵运算

改进分治算法的途径2:增加预处理

例子：平面点对问题

输入：平面点集 P 中有 n 个点, $n > 1$

输出： P 中的两个点，其距离最小

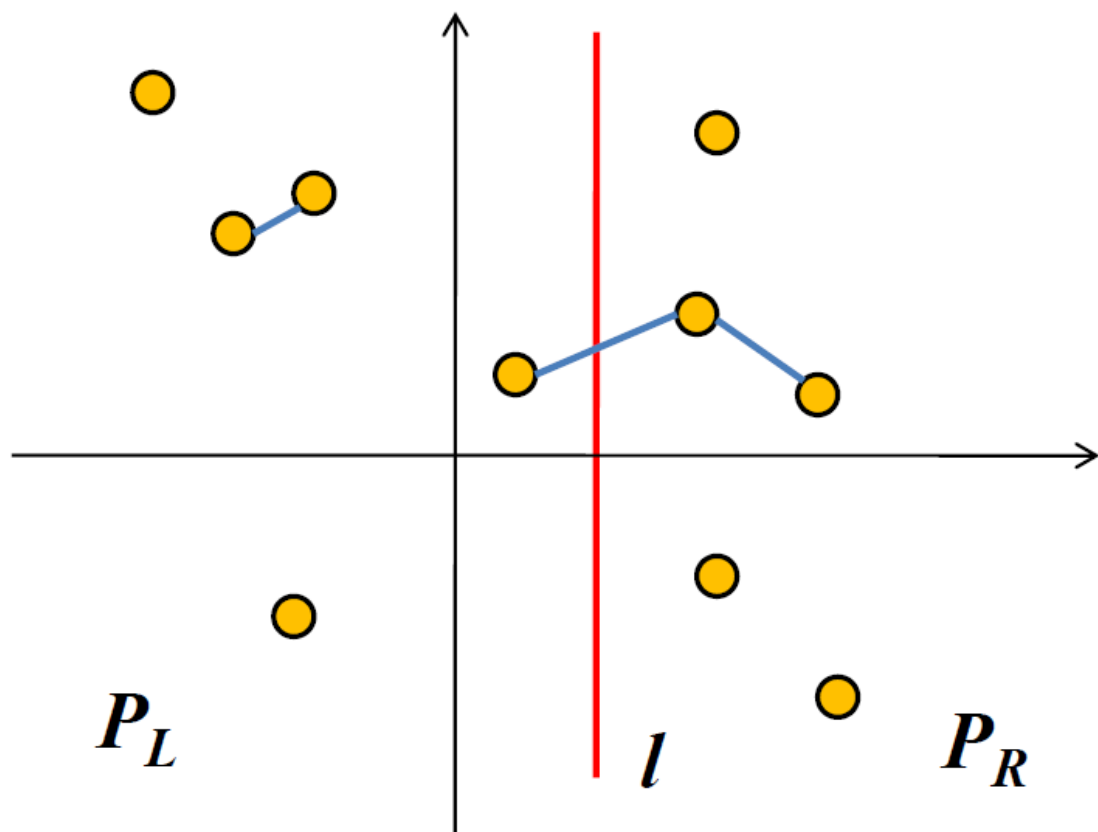
蛮力算法：

$C(n,2)$ 个点对, 计算最小距离, $O(n^2)$

分治策略： P 划为大小相等的 P_L 和 P_R

1. 分别计算 P_L 、 P_R 中最近点对
2. 计算 P_L 与 P_R 中各一个点的最近点对
3. 上述情况下的最近点对是解

划分实例： $n=10$



算法伪码

MinDistance (P, X, Y)

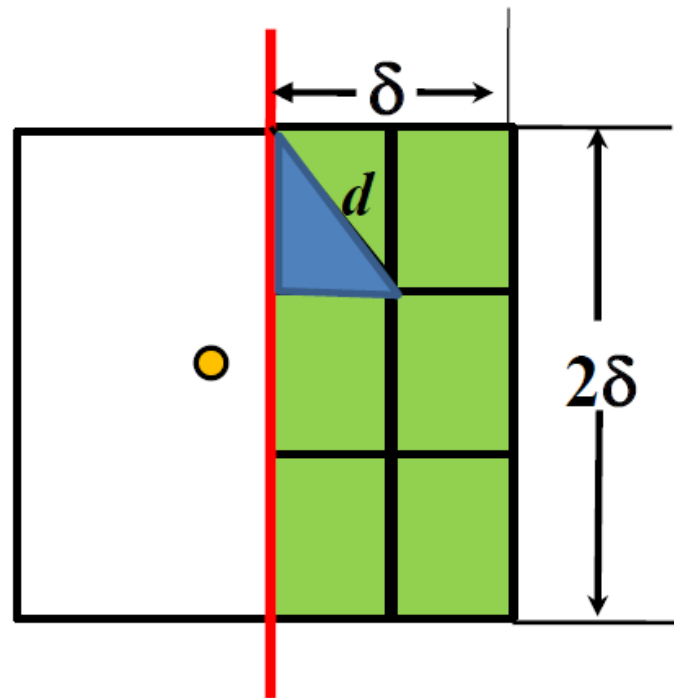
输入: 点集 P , X 和 Y 为横、纵坐标数组

输出: 最近的两个点及距离

1. 若 $|P| \leq 3$, 直接计算其最小距离
2. 排序 X, Y
3. 做中垂线 l 将 P 划分为 P_L 和 P_R
4. MinDistance (P_L, X_L, Y_L)
5. MinDistance (P_R, X_R, Y_R)
6. $\delta = \min(\delta_L, \delta_R)$ // δ_L, δ_R 为子问题的距离
7. 检查距 l 不超过 δ 两侧各1个点的距离. 若小于 δ , 修改 δ 为这个值

跨边界处理

$$\begin{aligned}d &= \sqrt{(\delta / 2)^2 + (2\delta / 3)^2} \\&= \sqrt{\delta^2 / 4 + 4\delta^2 / 9} \\&= \sqrt{25\delta^2 / 36} = 5\delta / 6\end{aligned}$$



右边每个小方格至多1个点，每个点至多比较对面的6个点，检查1个点是常数时间， $O(n)$ 个点需要 $O(n)$ 时间

算法分析

步1 递归边界处理: $O(1)$

步2 排序: $O(n\log n)$

步3 划分: $O(1)$

步4-5子问题: $2T(n/2)$

步6确定 δ : $O(1)$

步7检查跨边界点对: $O(n)$

$$T(n) = 2T(n/2) + O(n\log n)$$

$$T(n) = O(1), n \leq 3$$

递归树求解 $T(n) = O(n\log^2 n)$

增加预处理

原算法:

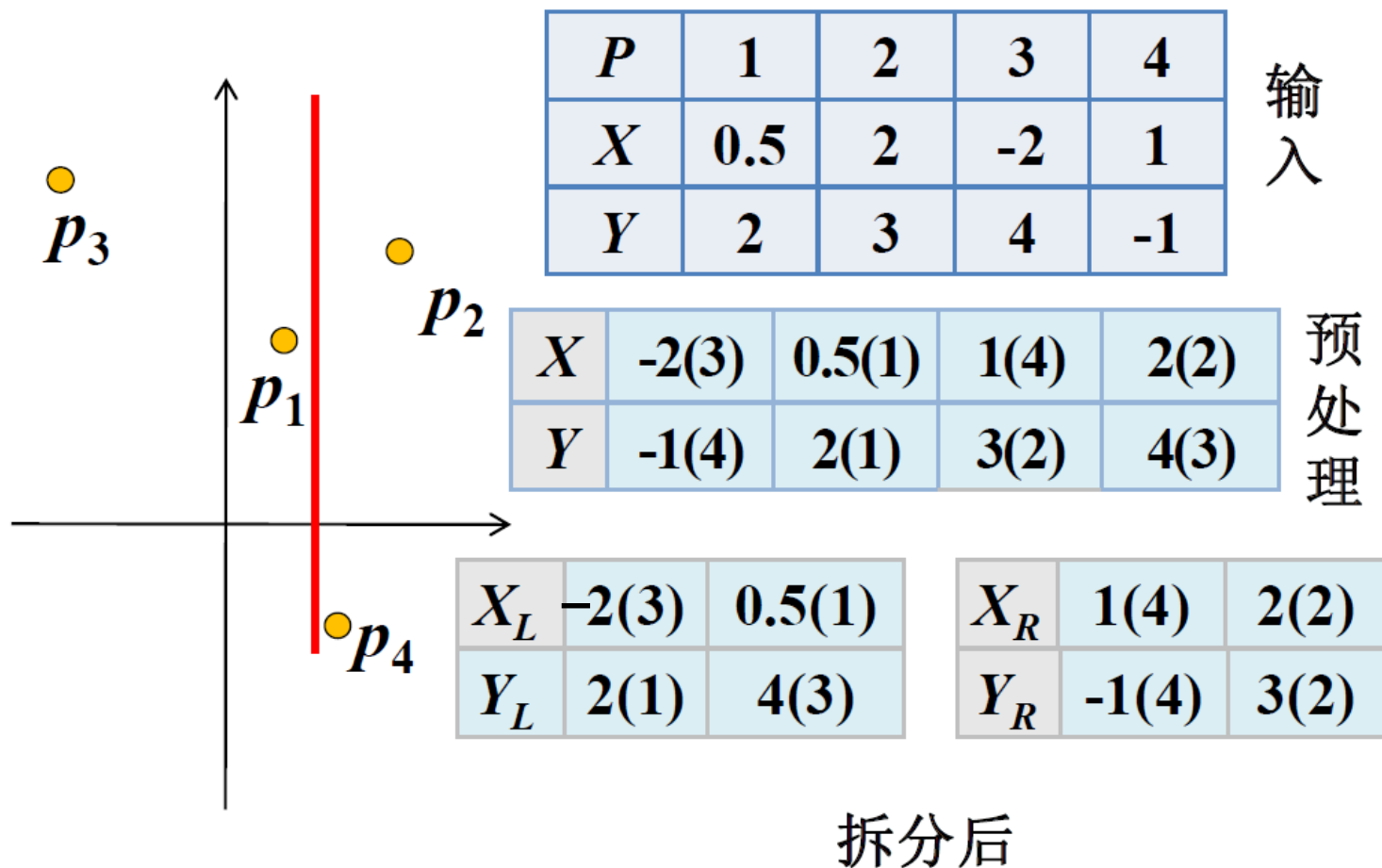
在每次划分时对子问题数组重新排序

改进算法:

1. 在递归前对 X, Y 排序, 作为预处理
2. 划分时对排序的数组 X, Y 进行拆分, 得到针对子问题 P_L 的数组 X_L, Y_L 及针对子问题 P_R 的数组 X_R, Y_R

原问题规模为 n , 拆分的时间为 $O(n)$

实例：递归中的拆分



改进算法时间复杂度

$W(n)$ 为算法时间复杂度

递归过程: $T(n)$, 预处理: $O(n\log n)$

$$W(n) = T(n) + O(n\log n)$$

$$T(n) = 2T(n/2) + O(n)$$

$$T(n) = O(1) \quad n \leq 3$$

解得

$$T(n) = O(n\log n)$$

于是

$$W(n) = O(n\log n)$$

典型实例分析

选择问题

输入：集合 L (含 n 个不等的实数)

输出： L 中第 i 小元素

$i=1$, 称为最小元素

$i=n$, 称为最大元素

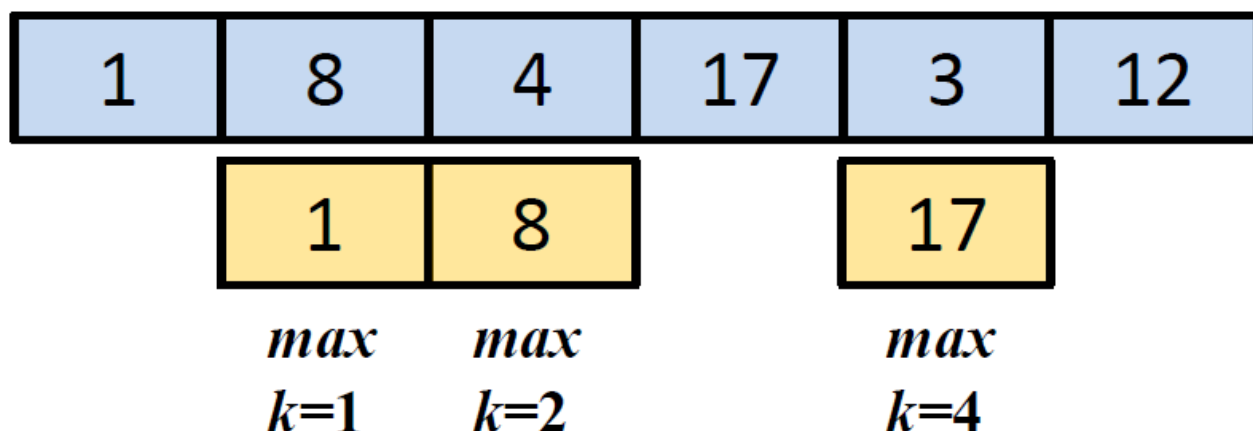
位置处在中间的元素，称为中位元素

n 为奇数，中位数唯一， $i = (n+1)/2$

n 为偶数，可指定 $i = n/2+1$

选最大

算法：顺序比较



输出： $max = 17$, $k=4$

算法最坏情况下的时间 $W(n)=n-1$

伪码

算法 Findmax

输入: n 个数的数组 L

输出: max, k

1. $max \leftarrow L[1]$
2. for $i \leftarrow 2$ to n do
3. if $max < L[i]$
4. then $max \leftarrow L[i]$
5. $k \leftarrow i$
6. return max, k

选最大最小

通常算法：

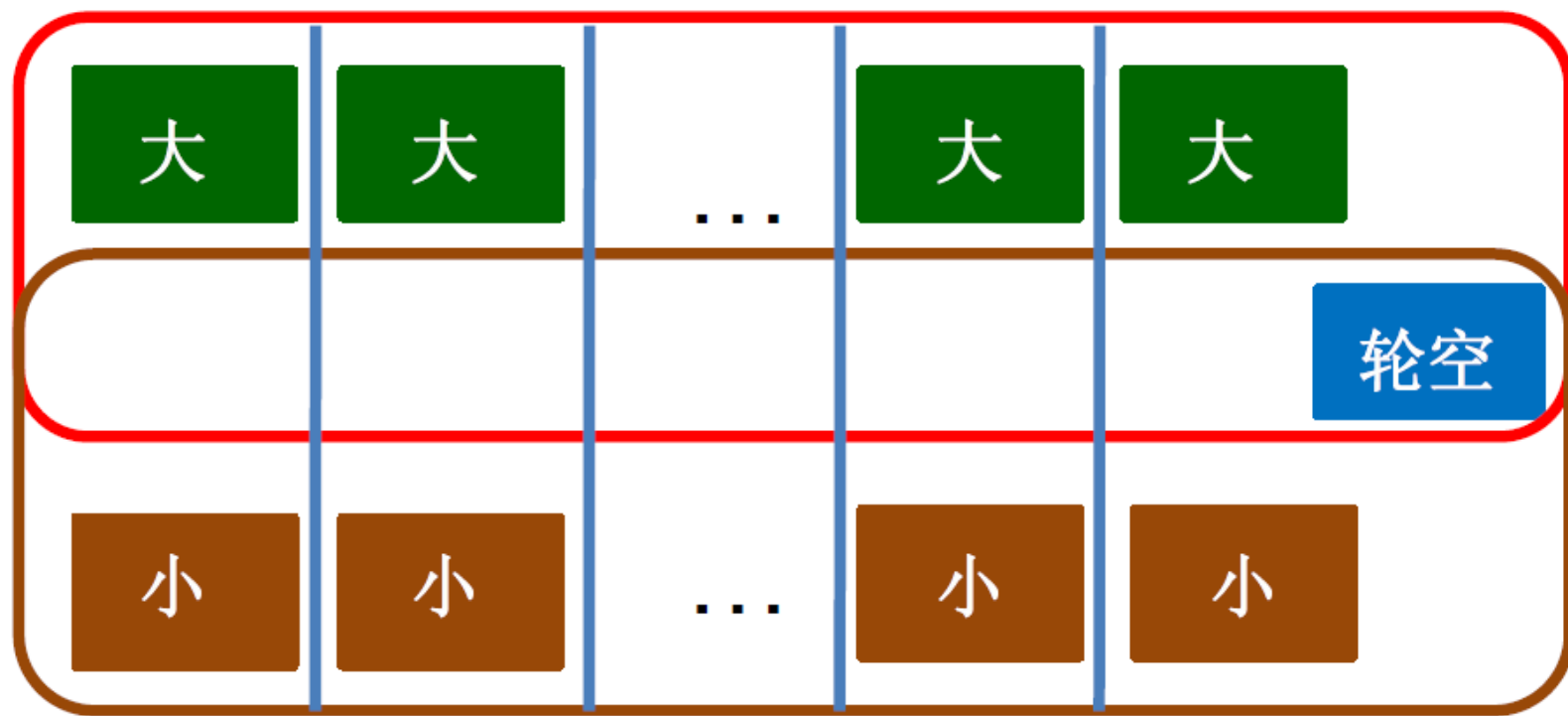
1. 顺序比较，先选最大 *max*
2. 顺序比较，在剩余数组中选最小 *min*，类似于选最大算法，但比较时保留较小的数

时间复杂性：

$$W(n) = n-1 + n-2 = 2n-3$$

分组算法

组1 组2 ... 组 $\lfloor n/2 \rfloor$



伪码

算法 FindMaxMin

输入: n 个数的数组 L

输出: max , min

1. 将 n 个元素两两一组分成 $\lfloor n/2 \rfloor$ 组
2. 每组比较, 得到 $\lfloor n/2 \rfloor$ 个较小和 $\lfloor n/2 \rfloor$ 个较大
3. 在 $\lceil n/2 \rceil$ 个较大 (含轮空元素) 中找最大 max
4. 在 $\lceil n/2 \rceil$ 个较小 (含轮空元素) 中找最小 min

最坏情况时间复杂度

行2 的组内比较: $\lfloor n/2 \rfloor$ 次

行3--4 求 *max* 和 *min* 比较:

至多 $2\lceil n/2 \rceil - 2$ 次

$$W(n) = \lfloor n/2 \rfloor + 2\lceil n/2 \rceil - 2$$

$$= n + \lceil n/2 \rceil - 2$$

$$= \lceil 3n/2 \rceil - 2$$

分治算法

1. 将数组 L 从中间划分为两个子数组 L_1 和 L_2
2. 递归地在 L_1 中求最大 max_1 和 min_1
3. 递归地在 L_2 中求最大 max_2 和 min_2
4. $max \leftarrow \max\{max_1, max_2\}$
5. $min \leftarrow \min\{min_1, min_2\}$

最坏情况时间复杂度

假设 $n = 2^k$,

$$W(n) = 2W(n/2) + 2$$

$$W(2) = 1$$

解
$$\begin{aligned} W(2^k) &= 2W(2^{k-1}) + 2 \\ &= 2[2W(2^{k-2}) + 2] + 2 \\ &= 2^2W(2^{k-2}) + 2^2 + 2 = \dots \\ &= 2^{k-1} + 2^{k-1} + \dots + 2^2 + 2 \\ &= 3 \cdot 2^{k-1} - 2 = 3n/2 - 2 \end{aligned}$$

选第二大

输入： n 个数的数组 L

输出： 第二大的数 $second$

通常算法： 顺序比较

1. 顺序比较找到最大 max
2. 从剩下 $n-1$ 个数中找最大，就是第二大 $second$

时间复杂度：

$$W(n) = n - 1 + n - 2 = 2n - 3$$

提高效率的途径

- 成为第二大数的条件：仅在与最大数的比较中被淘汰.
- 要确定第二大数，必须知道最大数.
- 在确定最大数的过程中记录下被最大数直接淘汰的元素.
- 在上述范围（被最大数直接淘汰的数）内的最大数就是第二大数.
- 设计思想： 用空间换时间.

锦标赛算法

1. 两两分组比较，大者进入下一轮，直到剩下 1 个元素 *max* 为止
2. 在每次比较中淘汰较小元素，将被淘汰元素记录在淘汰它的元素的链表上
3. 检查 *max* 的链表，从中找到最大元，即 *second*

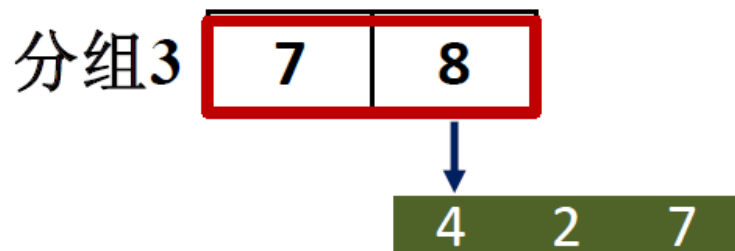
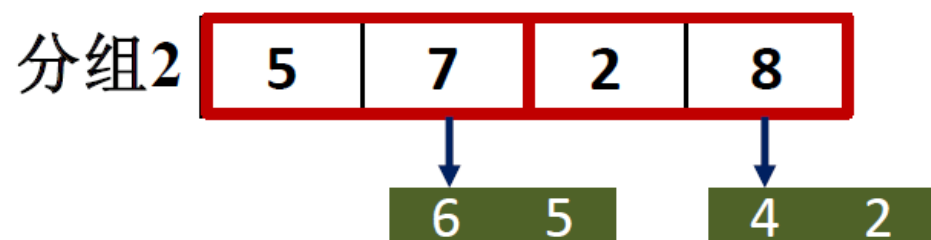
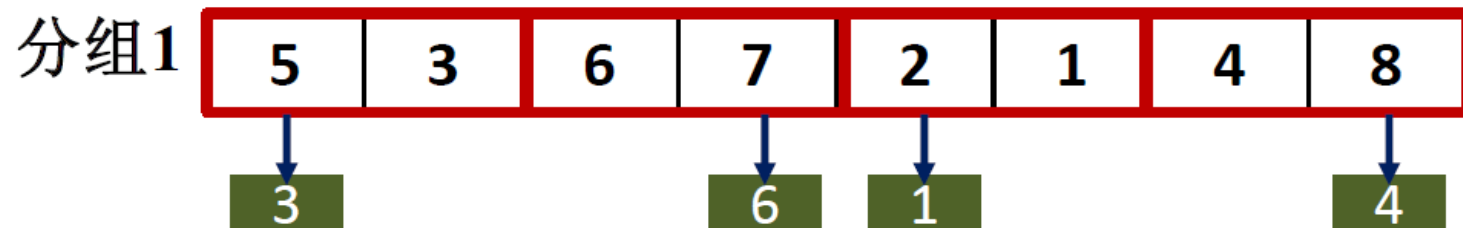
伪码

算法 FindSecond

输入: n 个数的数组 L , 输出: $second$

1. $k \leftarrow n$ // 参与淘汰的元素数
 2. 将 k 个元素两两1组, 分成 $\lfloor k/2 \rfloor$ 组
 3. 每组的2个数比较, 找到较大数
 4. 将被淘汰数记入较大数的链表
 5. if k 为奇数 then $k \leftarrow \lfloor k/2 \rfloor + 1$
 6. else $k \leftarrow \lfloor k/2 \rfloor$
 7. if $k > 1$ then goto 2
 8. $max \leftarrow$ 最大数
 9. $second \leftarrow max$ 的链表中的最大
- 一轮淘汰
- 继续分组淘汰

实例



时间复杂度分析

命题1 设参与比较的有 t 个元素，经过 i 轮淘汰后元素数至多为 $\lceil t/2^i \rceil$.

证 对 i 归纳. $i=1$, 分 $\lfloor t/2 \rfloor$ 组，淘汰 $\lfloor t/2 \rfloor$ 个元素，进入下一轮元素数是

$$t - \lfloor t/2 \rfloor = \lceil t/2 \rceil$$

假设 i 轮分组淘汰后元素数至多为 $\lceil t/2^i \rceil$ ，
那么 $i+1$ 轮分组淘汰后元素数为

$$\lceil \lceil t/2^i \rceil / 2 \rceil = \lceil t/2^{i+1} \rceil$$

时间复杂度分析（续）

命题2 max 在第一阶段分组比较中总计进行了 $\lceil \log n \rceil$ 次比较.

证 假设到产生 max 时总计进行 k 轮淘汰, 根据命题 1 有 $\lceil n/2^k \rceil = 1$.

若 $n=2^d$, 那么有

$$k = d = \log n = \lceil \log n \rceil$$

若 $2^d < n < 2^{d+1}$, 那么

$$k = d + 1 = \lceil \log n \rceil$$

时间复杂度分析（续）

第一阶段元素数： n

比较次数： $n-1$

淘汰了 $n-1$ 个元素

第二阶段：元素数 $\lceil \log n \rceil$

比较次数： $\lceil \log n \rceil - 1$

淘汰元素数为 $\lceil \log n \rceil - 1$

时间复杂度是

$$\begin{aligned} W(n) &= n - 1 + \lceil \log n \rceil - 1 \\ &= n + \lceil \log n \rceil - 2. \end{aligned}$$

一般性选择问题

问题：选第 k 小.

输入：数组 S , S 的长度 n , 正整数 k ,
 $1 \leq k \leq n$.

输出：第 k 小的数

实例 1

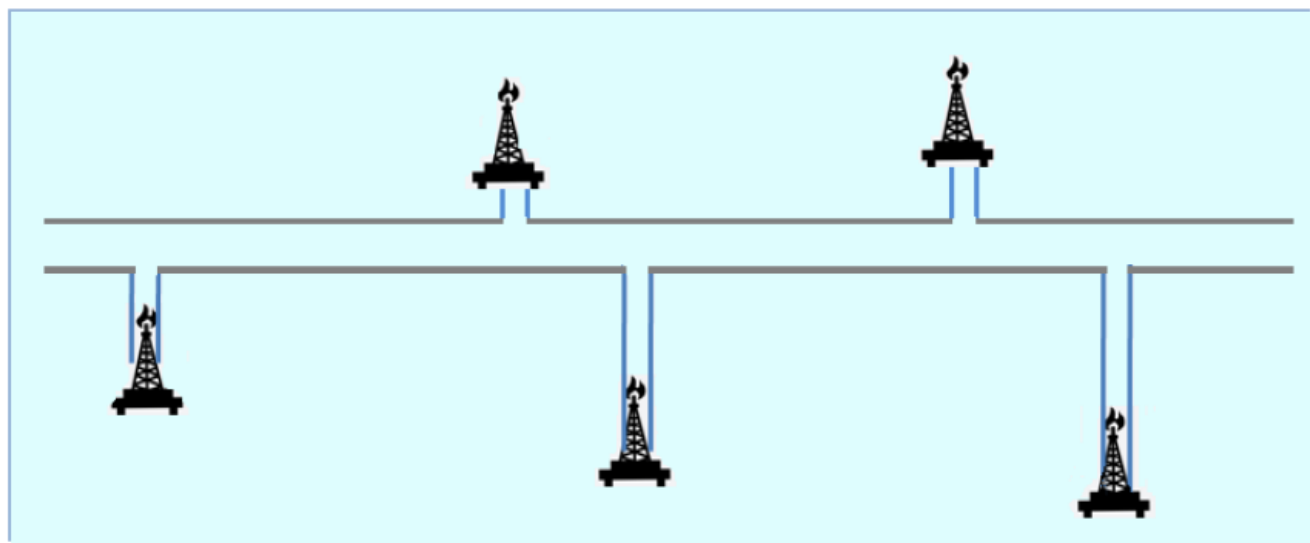
$S = \{ 3, 4, 8, 2, 5, 9, 18 \}$, $k = 4$, 解: 5

实例 2

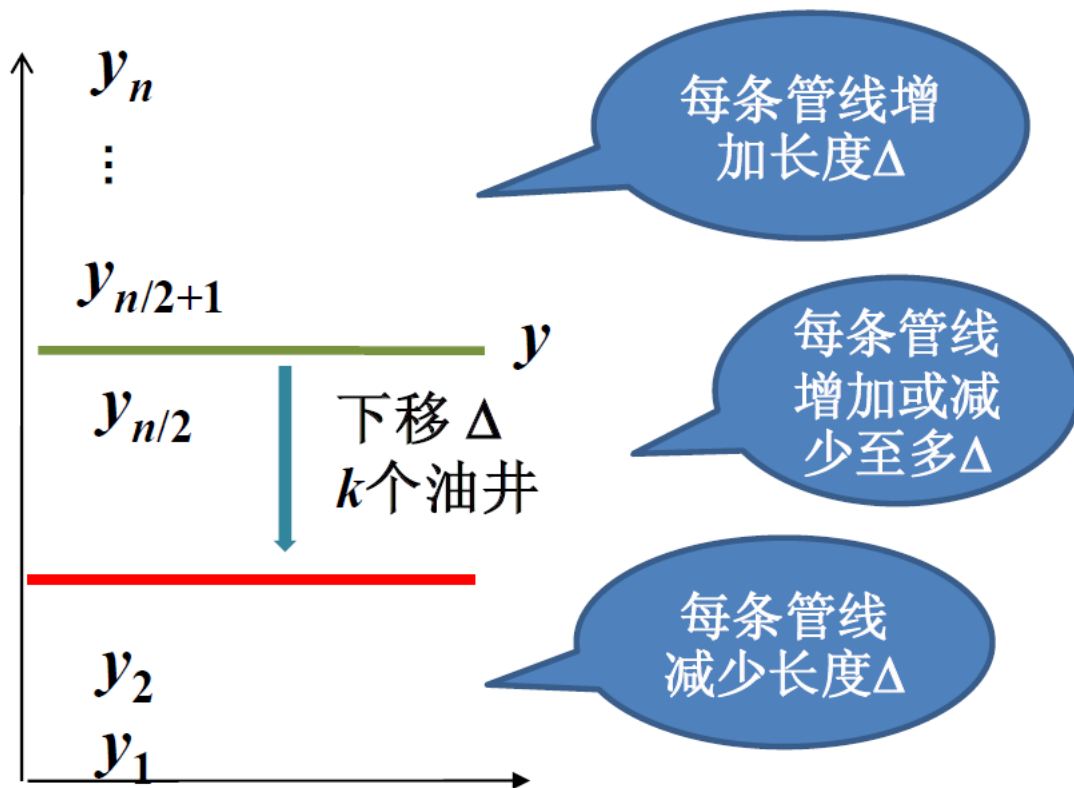
统计数据的集合 S , $|S|=n$,
选中位数, $k = \lceil n/2 \rceil$

一个应用：管道位置

问题：某区域有 n 口油井，需要修建输油管道. 根据设计要求，水平方向有一条主管道，每口油井修一条垂直方向的支管道通向主管道. 如何选择主管道的位置，以使得支管道长度的总和最小？



最优解: Y 坐标的中位数



下移后支管线总长度增加

简单的算法

算法一：

调用 k 次选最小算法

时间复杂度为 $O(kn)$

算法二：

先排序，然后输出第 k 小的数

时间复杂度为 $O(n \log n)$

分治算法

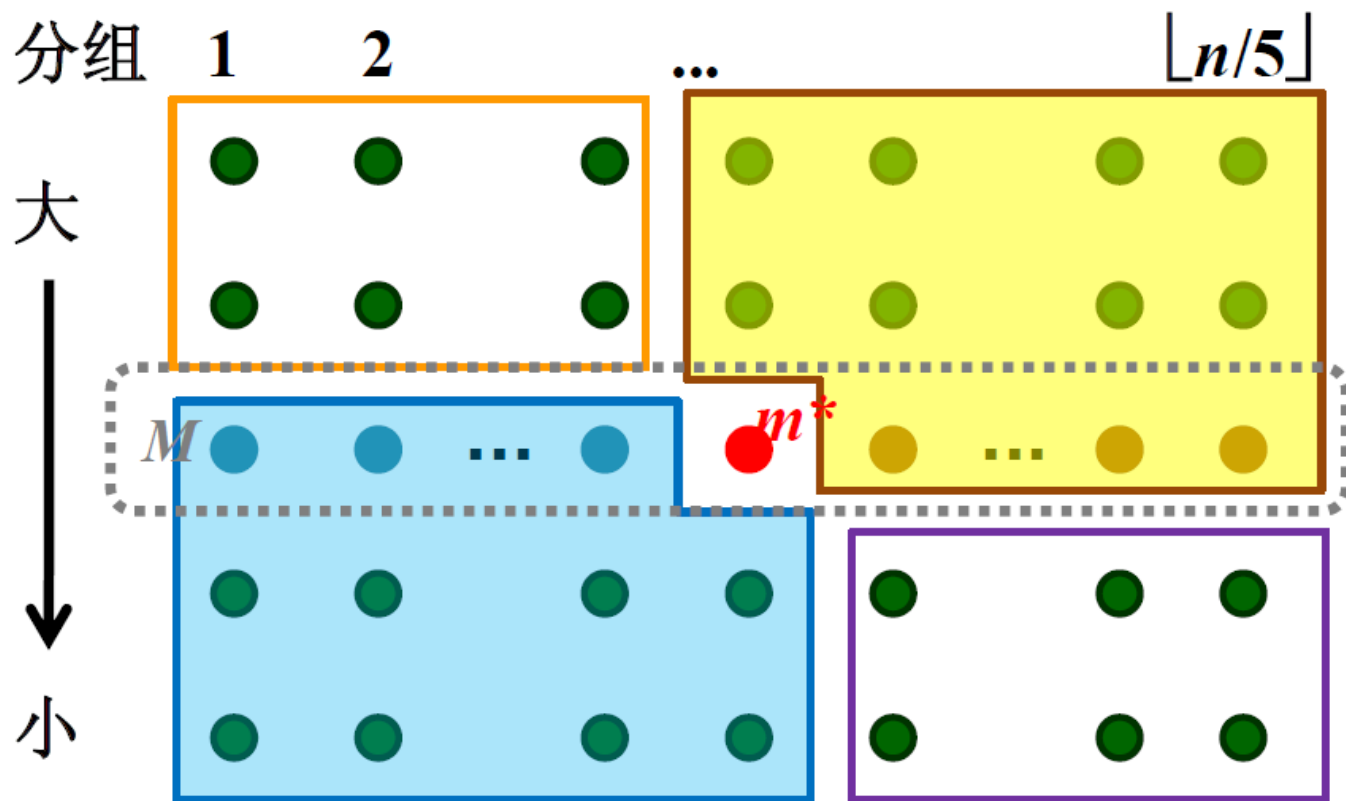
假设元素彼此不等，设计思想：

1. 用某个元素 m^* 作为标准将 S 划分成 S_1 与 S_2 ，其中 S_1 的元素小于 m^* ， S_2 的元素大于等于 m^* 。
2. 如果 $k \leq |S_1|$ ，则在 S_1 中找第 k 小。
如果 $k = |S_1| + 1$ ，则 m^* 是第 k 小
如果 $k > |S_1| + 1$ ，则在 S_2 中找第 $k - |S_1| - 1$ 小



算法效率取决于子问题规模，
如何通过 m^* 控制子问题规模？

m^* 的选择与划分过程



A: 数需要与 m^* 比大小, **B:** 数大于 m^*
C: 数小于 m^* , **D:** 数需要与 m^* 比大小

实例: $n=15, k=6$

8	2	3	5	7	6	11	14	1	9	13	10	4	12	15
---	---	---	---	---	---	----	----	---	---	----	----	---	----	----

	8	14	15	
	7	11	13	
M	5	9	12	$m^* = 9$
	3	6	10	
	2	1	4	
	8	14	15	
A	7	11	13	B
	5	9	12	
C	3	6	10	
	2	1	4	D

8, 7, 10, 4 需要与9比较

归约为子问题

S_1	8	14	15	S_2
	7	11	13	
	5	9	12	
	3	6	10	
	2	1	4	

子问题

8 7 5 3 2 6 1 4

子问题规模 = 8, $k=6$

伪码

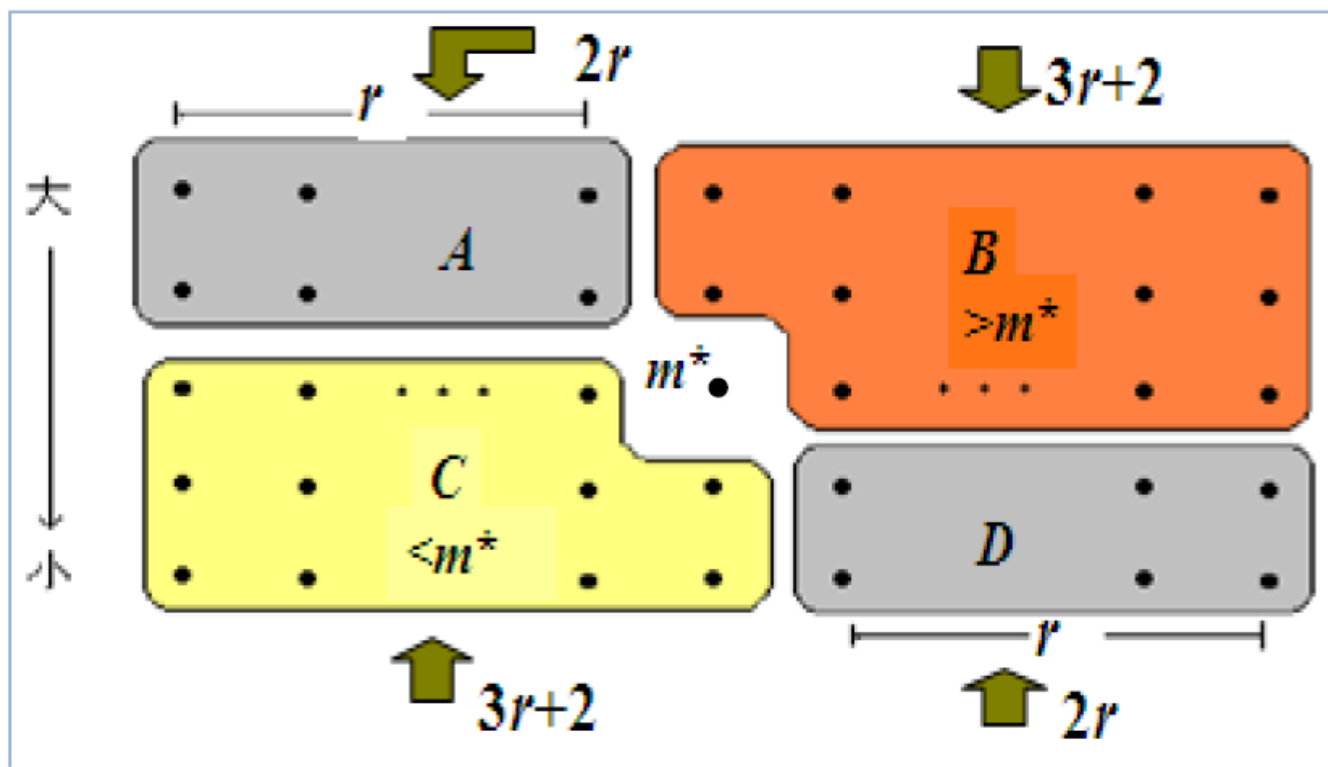
算法 $\text{Select}(S, k)$

输入：数组 S ，正整数 k ，

输出： S 中的第 k 小元素

1. 将 S 分 5 个一组，共 $n_M = \lceil n/5 \rceil$ 组
2. 每组排序，中位数放到集合 M
3. $m^* \leftarrow \text{Select}(M, \lceil |M|/2 \rceil)$ // S 分 A, B, C, D
4. A, D 元素小于 m^* 放 S_1 ，大于 m^* 放 S_2
5. $S_1 \leftarrow S_1 \cup C; S_2 \leftarrow S_2 \cup B$ 划分
6. if $k = |S_1| + 1$ then 输出 m^*
7. else if $k \leq |S_1|$ $\Leftarrow$ 递归计算子问题
8. then $\text{Select}(S_1, k)$
9. else $\text{Select}(S_2, k - |S_1| - 1)$

用 m^* 划分



$$n = 5(2r + 1), \quad |A| = |D| = 2r$$

子问题规模至多: $2r + 2r + 3r + 2 = 7r + 2$

子问题规模估计

不妨设 $n = 5(2r + 1)$, $|A|=|D|=2r$,

$$r = \frac{n/5 - 1}{2} = \frac{n}{10} - \frac{1}{2}$$

划分后子问题规模至多为

$$\begin{aligned} \underline{7r + 2} &= 7\left(\frac{n}{10} - \frac{1}{2}\right) + 2 \\ &= \frac{7n}{10} - \frac{3}{2} < \frac{7n}{10} \end{aligned}$$

时间复杂度递推方程

算法工作量 $W(n)$

行2: $O(n)$ //每5个数找中位数,构成 M

行3: $W(n/5)$ // M 中找中位数 m^*

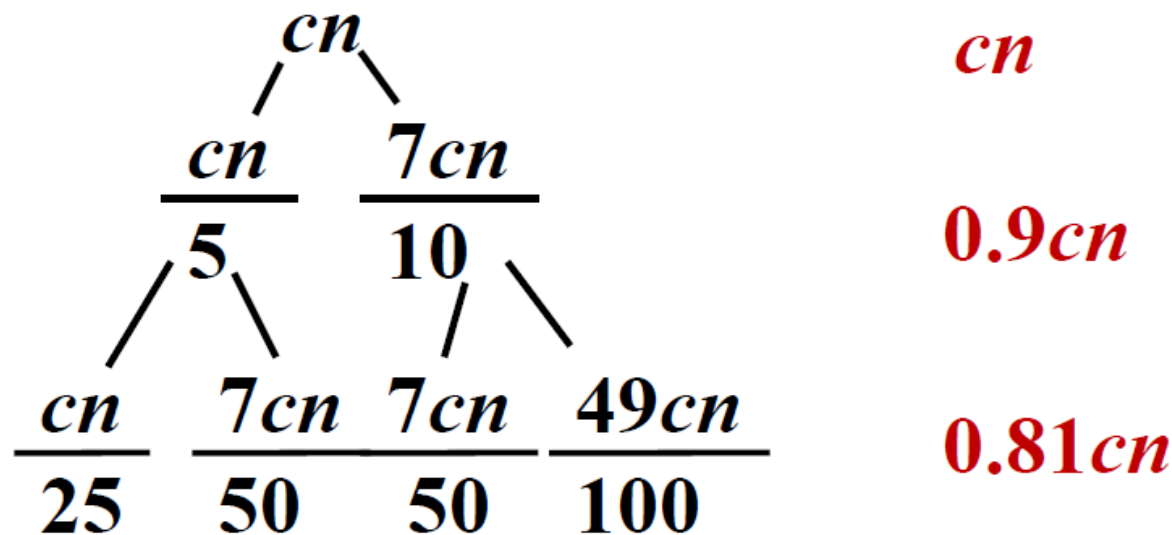
行4: $O(n)$ // 用 m^* 划分集合 S

行8-9: $W(7n/10)$ //递归

$$W(n) \leq W(n/5) + W(7n/10) + O(n)$$

递归树

$$W(n) = W(n/5) + W(7n/10) + cn$$



.....

$$W(n) \leq cn (1 + 0.9 + 0.9^2 + \dots) = O(n)$$

讨论



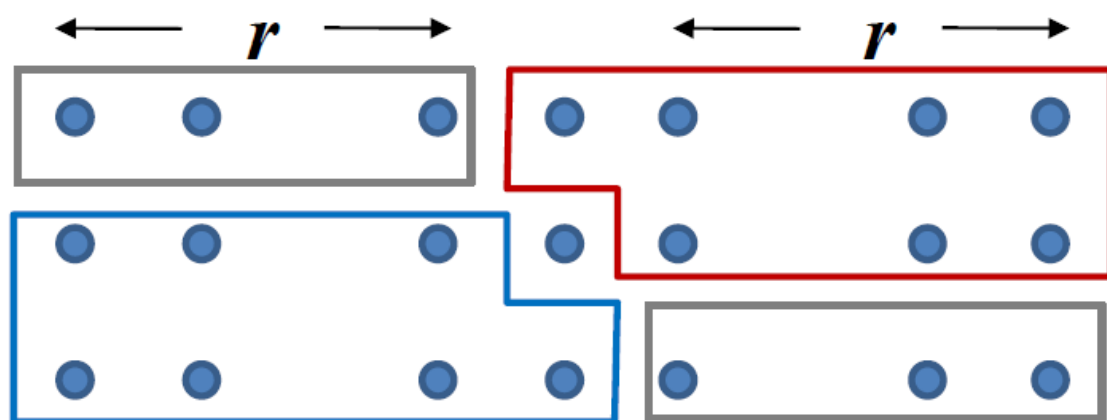
分组时为什么5个元素一组？
3个一组或7个一组行不行？

分析：递归调用

1. 求 m^* 的工作量与 $|M| = n/t$ 相关， t 为每组元素数. t 大, $|M|$ 小
2. 归约后子问题大小与分组元素数 t 有关. t 大, 子问题规模大

3分组时的子问题规模

假设 $t=3$, 3个一组:



$$n = 3(2r + 1)$$

$$r = (n/3 - 1)/2 = n/6 - 1/2$$

子问题规模最多为 $4r+1 = 4n/6 - 1$

算法的时间复杂度

算法的时间复杂度满足方程

$$W(n) = W(n/3) + W(4n/6) + cn$$

由递归树得 $W(n) = \Theta(n \log n)$

关键：

$|M|$ 与归约后子问题规模之和小于 n ，
递归树每行的工作量构成公比小于 1
的等比级数， 算法复杂度才是 $O(n)$ 。

1 的 $2n$ 次根

$$\omega_j = e^{\frac{2\pi j}{2n}i} = e^{\frac{\pi j}{n}i} = \cos \frac{\pi j}{n} + i \sin \frac{\pi j}{n} \quad j=0,1,\dots,2n-1$$

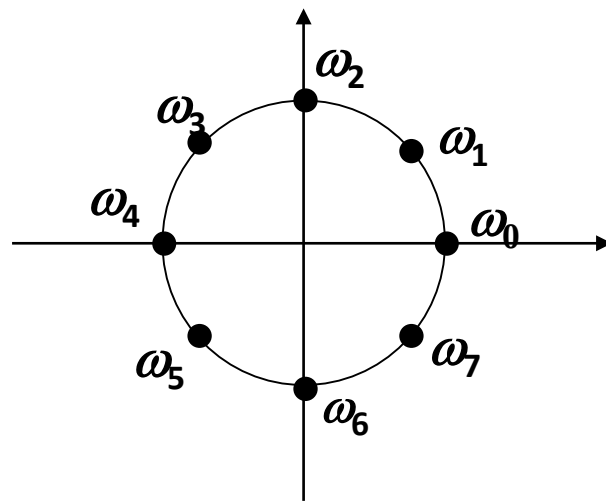
例如 $n=4$, 1的8次方根是:

$$\omega_0 = 1, \quad \omega_1 = e^{\frac{\pi i}{4}} = \frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}i,$$

$$\omega_2 = e^{\frac{\pi i}{2}} = i, \quad \omega_3 = e^{\frac{3\pi i}{4}} = -\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}i,$$

$$\omega_4 = e^{\pi i} = -1, \quad \omega_5 = e^{\frac{5\pi i}{4}} = -\frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}i,$$

$$\omega_6 = e^{\frac{3\pi i}{2}} = -i, \quad \omega_7 = e^{\frac{7\pi i}{4}} = \frac{\sqrt{2}}{2} - \frac{\sqrt{2}}{2}i$$



多项式求值

给定多项式: $A(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}$

设 x 为 1 的 $2n$ 次方根, 对所有的 x 计算 $A(x)$ 的值.

算法1: 对每个 x 做下述运算:

依次计算每个项 $a_i x^i$, 对 i 求和得到 $A(x)$,

$$T_1(n)=O(n^3)$$

算法 2.12

输入: $n-1$ 次多项式 A 的系数 $a_0, a_1, \dots, a_{n-1}$

输出: $A(\omega_0), A(\omega_1), \dots, A(\omega_{2n-1})$

1. 计算 $\omega_0, \omega_1, \dots, \omega_{2n-1}$

2. for $j \leftarrow 0$ to $2n-1$ do

$$3. \quad v \leftarrow a_0$$

4. for $i \leftarrow 1$ to $n-1$ do

5. $t \leftarrow a_i \cdot \omega_j^i$ // 计算 $A(x)$ 的 i 次项在 ω_j 的值

6. $v \leftarrow v + t$

```

7.   return v

```

多项式求值

给定多项式: $A(x) = a_0 + a_1x + a_2x^2 + \dots + a_{n-1}x^{n-1}$

设 x 为 1 的 $2n$ 次方根, 对所有的 x 计算 $A(x)$ 的值.

算法2: $A_1(x) = a_{n-1}$

$$A_2(x) = a_{n-2} + xA_1(x)$$

$$A_3(x) = a_{n-3} + xA_2(x)$$

...

$$A_n(x) = a_0 + xA_{n-1}(x) = A(x)$$

对每个 x 按照上述顺序求值

$$T_2(n) = O(n^2)$$

分治算法

原理:

$$A_{\text{even}}(x) = a_0 + a_2x + a_4x^2 + \dots + a_{n-2}x^{(n-2)/2}$$

$$A_{\text{odd}}(x) = a_1 + a_3x + a_5x^2 + \dots + a_{n-1}x^{(n-2)/2}$$

$$A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2), \quad x^2 \text{ 为 } 1 \text{ 的 } n \text{ 次根}$$

算法3:

1. 计算1 的所有的 $2n$ 次根
2. 分别计算 $A_{\text{even}}(x^2)$ 与 $A_{\text{odd}}(x^2)$
3. 利用步2 的结果计算 $A(x)$

复杂度分析: $T(n) = T_1(n) + f(n)$, $f(n) = O(n)$ 计算 $2n$ 次根时间

$$T_1(n) = 2T_1(n/2) + g(n), \quad g(n) = O(n),$$

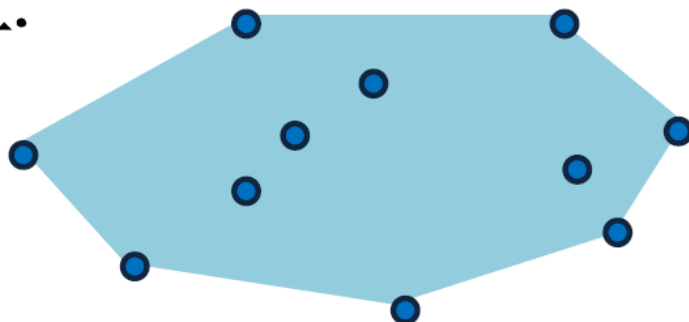
$$T_1(1) = O(1)$$

$$T(n) = O(n \log n)$$

平面点集的凸包

问题（平面点集的凸包）

给定大量离散点的集合 Q ，求一个最小的凸多边形，使得 Q 中的点在该多边形内或者边上。

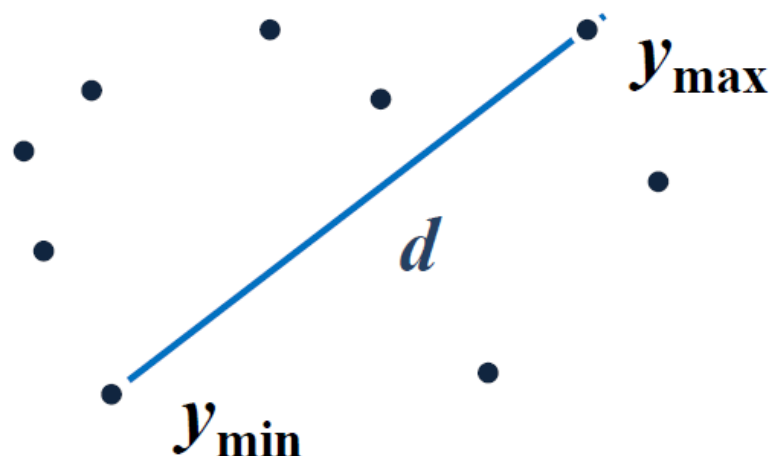


应用背景

图形处理中用于形状识别：字形识别、碰撞检测等

分治算法

1. 以连接最大纵坐标点 $y_{\max}$ 和最小纵坐标点 $y_{\min}$ 的线段 $d = \{y_{\max}, y_{\min}\}$ 划分 L 为左点集 L_{left} 和右点集 L_{right}



2. Deal (L_{left}); Deal (L_{right})

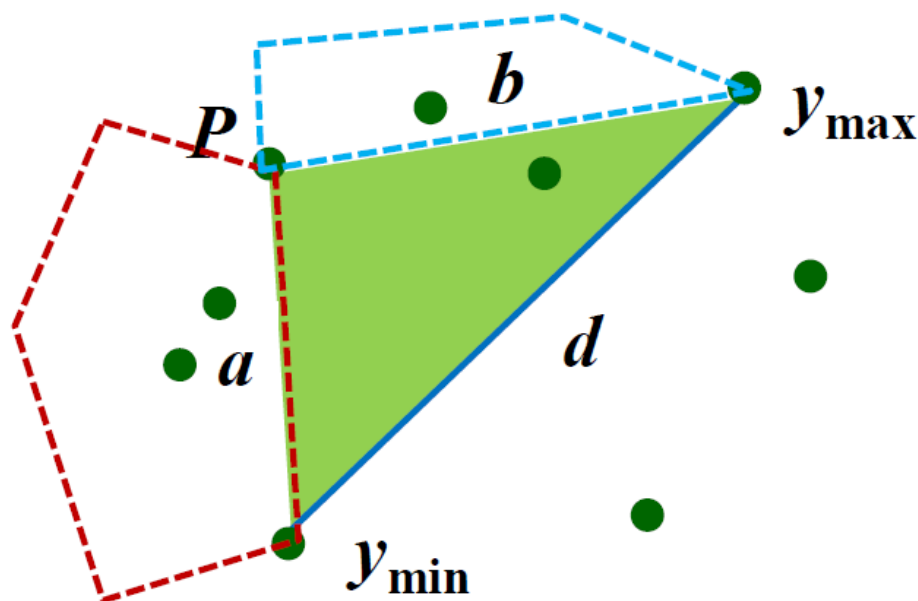
Deal (L_{left})

考虑 L_{left} : 确定距 d 最远的点 P

在三角形内的点, 删除;

a 外的点与 a 构成 L_{left} 的子问题;

b 外的点与 b 构成 L_{left} 的子问题.



伪码

Deal (L_{left})

1. 以 d 和距离 d 最远点 P 构成三角形, P 加入凸包, 另外两条边分别记作 a 和 b
2. 检查 L_{left} 中其他点是否在三角形内; 在则从 L 中删除; 否则根据在 a 或 b 边的外侧划分在两个子问题中
3. **Deal (a)**
4. **Deal (b)**

算法分析

- 初始用 d 划分 $O(n)$
- Deal 递归调用 $W(n)$
 - 找凸包顶点 P $O(n)$
 - 根据点的位置划分子问题 $O(n)$

- $$W(n) = W(n-1) + O(n)$$
$$W(3) = O(1)$$

最坏情况为 $O(n^2)$

$$T(n) = O(n) + W(n) = O(n^2)$$

- Graham扫描算法 $O(n \log n)$

上机作业

对于给定的 N 本书和 M 个学生，每本书的页数已经按升序排列。我们的任务是分配这些书，使得分配给每个学生的最大阅读页数达到最小，并且每个学生需要阅读升序排列上连续的书。输出最小页数。

例子：

输入： $N=4$ ， 页数[] = {12, 34, 67, 90}, $M = 2$

输出： 113。

- 提交要求：介绍算法实现思想（流程图、公式辅助说明）、输出截图、附上源代码，提交pdf文档。
- 提交方式：65959505@qq.com
- Pdf命名格式：学号_姓名__第几次作业
如第一次作业格式：203838_张三_01
- 请各位同学10月14日18点前交作业，逾期无效